

About

Contents

Complying with licenses	6
What are licenses?	6
Requirements	7
Inclusion	7
Third-party licenses	8
Credits	8
Documentation changelog	10
New pages since version 2.0	10
Frequently asked questions	10
General	10
Troubleshooting	11
Godot Orchestrator Docs	11
Get involved	11
Offline documentation	11
Introduction	12
Before you start	12
About Orchestrator	12
Organization of the documentation	12
About this documentation	13
Release policy	13
Versioning	13
Release support timeline	14
Which version should I use for a new project?	15
When is the next release out?	15
What are the criteria for compatibility across versions?	16
System requirements	17
Orchestrator	17

Exported Orchestrator project	17
Tutorials and resources	17
Tutorials	17
Showcase Videos	17
Contribution workflow	18
Pull request workflow	18
Git source repository	18
Forking and cloning	18
Rebasing changes on conflicts	19
Code style guidelines	20
C++ and Objective-C	20
File names	20
Header includes	20
Constants and enums	22
Comment style guide	23
C++ usage guidelines	23
Rationale	23
Disallowed features	24
Standard Template Library	24
auto keyword	24
Complete script	45
Refactor and use functions	45
Moving apply rotation to a function	45
Moving apply movement to a function	46
Listening for player input	46
Turning	47
Moving forward	50
Summary	53
All nodes	53
Arrays	54
Overview	55
Creating an array	55
Node properties	56
Add element	56
Node properties	56
Find element index	56
Node properties	56
Remove element	57
Remove by index	57
Remove by value	57
Clear array	58

Append arrays	58
Operators	59
Get element at index	59
Set element at index	59
Autoloads	60
Godot's autoload concepts	60
Registering autoloads	61
Accessing autoloads in orchestrations	61
Changing the autoload reference	61
Orchestrations as autoloads	61
Comments	62
Creating comment nodes	62
Editing comment node properties	62
Deleting comment nodes	62
Attach/Detach nodes	63
Auto-shrink	63
Properties	63
Constants	64
Global constants	64
Math-specific constants	65
Class-specific constants	65
Singleton-specific constants	66
Type-specific constants	66
Dialogue	67
Show message node	67
Using a custom scene	67
Non-linear conversation flow	68
Properties	68
Dialogue choice node	69
Dictionaries	69
Overview	69
Creating a dictionary	70
Node properties	70
Setting value in an existing dictionary	70
Node properties	71
Events	71
Init event	72
Notification event	72
Enter tree event	72
Exit tree event	72
Input event	73

Physics process event	73
Process event	73
Ready event	73
Shortcut input event	74
Unhandled input event	74
Unhandled key input event	74
Custom Events	74
Flow Control	74
Branch - If/Else	75
Node properties	75
Chance	75
Node properties	75
Delay	76
Using delay in functions	76
Changing pin types	81
Node properties	82
Sequence	82
Node properties	82
Switch nodes	82
General Switch	83
Enumeration Switch	83
Integer Switch	84
String switch	84
Type cast - Is	85
Changing the selected cast type	85
Node properties	86
While	86
Node properties	86
Functions	87
Function Types	87
Function access specifiers	88
Declaring Script Functions	88
Creating a function	88
Editing a function	88
Coding	89
Calling functions	89
Variable argument functions	90
Method chaining	90
Input	91
Defining input mappings	91
Using an input mapping	92
Setting the input action name	92
Input action trigger modes	92

Input action properties	93
Math	93
Operators	93
Legacy non-promotable operators	94
Promotable operators	94
Memory	95
Creating objects	96
Node Properties	96
Freeing objects	96
Node Properties	97
Properties	97
Property types	97
Accessing properties	97
Get property	98
Set property	98
Properties via Methods	98
Resources	98
Resource paths	99
Node properties	99
Preload resources	99
Scenes	100
Get self	100
Scene nodes	100
Accessing scene node behaviors	101
Node properties	102
Scene tree	102
Using the get_tree method	102
Using the get scene tree node	103
Node properties	103
Instantiating scenes	103
Set what scene to load	103
Node properties	103
Signals	104
Creating signals	104
Renaming signals	104
Deleting a signal	104
Passing data with signals	105
Emit a signal	105
Emitting a local signal	105
Emitting an external signal	106
Changing signal's arguments	106

Await a signal	106
How to await a signal	106
Node properties	107
Singletons	107
Singleton types	107
Selecting the singleton	108
Accessing singleton functions	109
Registering a user-defined singleton	109
Utilities	110
Make built-in type nodes	110
Breaking built-in type nodes	110
Print string	111
Node properties	111
Variables	111
Variable Types	111
Built-in types	111
Arrays and dictionaries	114
Exported variables	114
Private variables	114
Variable descriptions	115
Promoting to variable	115
Accessing variables	115
Editing variables	115
Renaming variables	116
Variable properties	116
Getting and setting variable values	116
Get nodes	116
Set nodes	117
Shortcuts when dragging	117

Complying with licenses

What are licenses?

Orchestrator is created and distributed under the Apache License, Version 2.0 license. The project does not have a sole owner, as every contributor that submits code to the project does so under the same license and keeps ownership of their contributions.

The license is the legal requirement for you (or your company) to use and distribute the software (and derivative works, including games made with it). Your game or project can have a different license, but it still needs to comply with the original one.

In your project's credits screen, remember to include third-party notices for assets you're using, such as textures, models, sounds, music, and fonts. Free assets in particular often come with licenses that require attribution. Double-check their licenses before using those in public projects.

Requirements

For derived works that include Orchestrator, but do not modify or change its source, include the following:

This game uses Orchestrator, which is available under the following license:

Copyright (c) 2023-present Crater Crash Studios, LLC and it's contributors.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

For works that not only include Orchestrator, but includes a modified version of the plug-in, be sure you comply with the full Apache License, Version 2.0 requirements. These requirements not only require the inclusion of the above text, but all source files of the original work must retain the original copyright and license preamble. Additionally, any changes made to the source must be recorded in the headers of the source files. While changes are not required to be submitted back to the original project, they're welcomed!

Your game does not need to be under the same license as Orchestrator. You are free to release your Godot project under any license, and to create commercial games with this plug-in.

Inclusion

The license does not specify specifically how it must be included in your game. There are a number of options that you can freely use depending on how you distribute or wish to share license information with your users. These are the most common approaches, and you only need to implement one, not all.

Credits screen Include the above License text somewhere in the credits screen. It can be at the bottom after showing the rest of the credits. Most studios use this approach with open source licenses.

Licenses screen Some games have a special menu (often in the settings) to display licenses. This menu is typically accessed with a button called **Third-party Licenses** or **Open Source Licenses**. If your game uses such a style, include the above License text there.

Accompanying file If the game is distributed on desktop platforms, a file that contains the License text can be added to the software that is installed on the user's PC.

Printed manual If the game includes printed manuals, License text can be included there.

Third-party licenses

Orchestrator is free and open-source, and its built using other free and open-source technologies. Each of those technologies have their own respective licenses, and thanks to them, Orchestrator is possible.

Godot GDExtension (godot-cpp)

Copyright (c) 2017-present Godot Engine contributors.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Credits

These are assets used by this documentation, but are from other sources.

Image Names	Attribution	License
key_concepts_character_icons	Julian Linietsky, Ariel Manzur and the Godot community.	LICENSE.txt
key_concepts_signals	Julian Linietsky, Ariel Manzur and the Godot community.	LICENSE.txt

Documentation changelog

The documentation is continually being improved. New releases include new pages, fixes, and updates to existing pages, and many updates to the Script Node Reference. Below is a list of new pages added since version 2.0.

This document only contains new pages so not all changes are reflected.

New pages since version 2.0

- Added Memory nodes for Creating Objects and Freeing Objects details.
- Added Method Chaining details.

Frequently asked questions

General

- **What license is Orchestrator code under?** Orchestrator is licensed under the Apache License, Version 2.0.
- **What license is Orchestrator logo and marketing images under?** The Orchestrator logo and marketing images, including this website, are Copyright © 2023 Crater Crash Studios. All Rights Reserved.
- **Do you offer support for this plugin?** Yes, we are committed to providing support for our plugin, simply leave your questions on our discord.
- **Where can I find the source code for Orchestrator?** You can find the source code on our GitHub repository.
- **I have a suggestion or idea, how do I provide this to you?** Feel free to leave it in our discord in the `#suggestions` channel, or our GitHub repository.
- **Are you affiliated with Godot or the Godot Engine?** No, Crater Crash Studios is not affiliated with Godot or the Godot Engine.

- **Does Orchestrator support Godot 4.2 and above?** Yes, Orchestrator supports Godot 4.2 and above. It does not work with any version of Godot prior to 4.2.

Troubleshooting

- ****Error: “Required virtual method ScriptExtension::_get_language must be overridden before calling.”**** This is an unfortunate bug in the Godot engine, and it’s normal for GDExtension plug-ins. This will have no impact on the plug-in’s stability, so it’s safe to ignore.

Godot Orchestrator Docs

Orchestrator’s documentation is available in various languages and versions. Expand the language or version selector in the top navigation panel to see the possible values.

Welcome to the official documentation of Orchestrator, the ultimate Visual Scripting solution specific designed for Godot platform. If you are new to this documentation, we recommend that you read the introduction page to get an overview of what this documentation has to offer.

The table of contents in the sidebar should let you easily access the documentation for your topic of interest.

Get involved

Orchestrator is an open source project developed by Crater Crash Studios, LLC in collaboration with a community of volunteers. The documentation team can always use feedback and help to improve tutorials, class reference guides, and more. If you don’t understand something, or cannot find what you are looking for in the docs, help us make the documentation better by letting us know!

Submit an issue or pull request on the GitHub repository or help translate the documentation into other languages.

Offline documentation

To browse the documentation offline, you can clone the GitHub repository.

This documentation is built using Docusaurus, using React and Node.JS. Simply install the Node.JS package manager **npm**, and you can easily build and serve this documentation locally.

Introduction

Welcome to the official documentation of **Orchestrator**, the ultimate Visual-Scripting plug-in for Godot. Orchestrator is a powerful, yet user-friendly plug-in that you can use to develop any type of game.

This page gives a broad overview of the plug-in and this documentation, so you know where to start if you are a beginner or where to look should you need more specific information on a feature.

Before you start

The Tutorials and resources page has a list of video tutorials contributed by various authors, including the authors of the plug-in. If you prefer video to text, consider checking them out. Otherwise, Getting Started is a great starting point.

In case you have trouble with one of the tutorials or your project, you can always find help through the Crater Crash Studios Discord community, our Forums, or the Godot Discord.

About Orchestrator

Every game emerges from a single idea, and the creation of a game involves many moving parts. Orchestrator's goal is to provide a series of easy to use, highly customizable features to create game logic, interactions, and supporting logic using visual scripting rather than traditional text-based code.

Orchestrator is completely free and open source under the Apache License, Version 2.0. The game you create is yours, down to the last line of code. This plug-in's development is fully independent and entirely community-driven, giving the community the ability to shape the plug-in's future for their needs and expectations.

Organization of the documentation

This documentation is organized into several sections:

- **About** contains this introduction, as well as information about the plug-in, its history, licensing, authors, and more. It also contains the Frequently asked questions section, helping answer some of the most common questions.
- **Getting Started** contains all necessary information on using the plug-in to make games. It starts with a **Step by Step** tutorial which should be the entry point for all new users. **This is the best place to start if you're new!**
- The **Manual** can be read or referenced as needed, in any order. It contains feature-specific tutorials and documentation.

- **Contributing** provides information related to contributing back to Orchestrator, whether to the plug-in’s code, documentation, examples, or other aspects. It describes how to report bugs, how contributor workflows are organized, and more. It also has sections intended for advanced users and contributors, with information on compiling the plug-in.
- **Community** is dedicated to the life of the Orchestrator plug-in’s community. It refers to various community outlets and discords, and contains a list of recommended tutorials and materials beyond this documentation.
- The **Script Node Reference** documents each Orchestrator visual script node type, how you can use them, their purpose, and how to configure them. This is where you should check if you don’t understand a specific node or are unsure how to locate a specific node or feature.

In addition to this documentation, you may also want to take a look at the various Orchestrator example projects.

About this documentation

The Orchestrator community continuously writes, corrects, edits, and improves this documentation. We are always looking for more help. You can also contribute by opening GitHub issues or translating the documentation into your own language. If you are interested in helping, see Ways to contribute and Writing documentation, or get in touch with the team on Discord.

All documentation content is licensed under the permissive Creative Commons Attribution 3.0 (CC BY 3.0) license, with attribution to “*Crater Crash Studios, LLC and the community*” unless otherwise noted.

Have fun reading and making games with Orchestrator!

Release policy

Orchestrator’s release policy is constantly evolving. The description below provides a general idea of what to expect, but what may actually happen depends on multiple factors, including the needs of the community at any given moment.

Versioning

Orchestrator loosely follows Semantic Versioning with a `major.minor.patch` version system.

- The `major` version is incremented when major compatibility breakages happen which imply significant porting work to move projects from one major version to another.

For example, porting Orchestrator-based projects from Orchestrator v2.x to Orchestrator v3.x requires running the project through a conversion tool, and then performing potentially a number of further adjustments manually for what the tool could not do automatically.

- The **minor** version is incremented for feature releases that do not break compatibility in a major way. Minor compatibility breakage in very specific circumstances *may* happen in minor versions, but the vast majority of projects should not be affected or require significant porting work.

This is because Orchestrator provides features that cover a vast array of Godot game engine features, like rendering, physics, and scripting. Fixing bugs or implementing new features in one area could require changes to a feature's behavior or modifying some code aspect, even if the change remains mostly backward compatible.

Upgrading to a new minor version is recommended for all users, but some testing should be done to ensure that your projects behave as expected.

- The **patch**, or often referred to as maintenance or bugfix, version is incremented for maintenance releases which focus on fixing bugs, security issues, implementing new requirements, and backporting safe usable enhancements. Patch releases are backwards compatible.

Patch versions may include minor new features which do not impact the existing API, and thus have no risk of impacting existing projects.

Updating to new patch versions is therefore considered safe and strongly recommended to all users.

We call **major.minor** combinations *stable branches*. Each stable branch starts with a **major.minor** release without the 0 for **patch**) and is further developed for maintenance releases in a Git branch of the same name (for example patch updates for the 2.0 stable branch are developed in the 2.0 Git branch).

Release support timeline

Stable branches are supported *at least* until the next stable branch is released and has received its first patch update. In practice, we support stable branches on a *best effort* basis for as long as they have active users who need maintenance updates.

Whenever a new major version is released, we make the previous stable branch a long-term supported release, and do our best to provide fixes for issues encountered by users of that branch who cannot port complex projects to the new major version.

In a given minor release series, only the latest patch releases receives support. If you experience an issue using an older patch release, please upgrade to the latest patch release of that series and test again before reporting issues on GitHub.

Version	Release Date	Support Level
Orchestrator 2.6 (main)	Q4 2026 (estimate)	<i>Development.</i> Receives new features, usability and performance improvements, as well as bug fixes, while under development.
Orchestrator 2.5	July 2026	Receives new features, usability and performance improvements, as well as bug fixes.
Orchestrator 2.4	February 2026	Receives fixes for bugs and security issues, as well as patches that enable platform support.
Orchestrator 2.3	November 2025	No longer supported (last update: 2.3.7).
Orchestrator 2.2	November 2025	No longer supported (last update: 2.2.7).
Orchestrator 2.1	August 2024	No longer supported (last update: 2.1.5).
Orchestrator 2.0	March 2024	No longer supported (last update: 2.0.7).
Orchestrator 1.1	December 2023	No longer supported (last update: 1.1.0).
Orchestrator 1.0	August 2023	No longer supported (last update: 1.0.5).

Legend: Full support – Partial support – No support (end of life) – Development version

Pre-release Orchestrator plug-in versions aren't intended to be used in production and are provided for testing purposes only.

Which version should I use for a new project?

We recommend using Orchestrator 2.5 for all new projects.

When is the next release out?

The Orchestrator contributors do not work under any strict deadlines, but we strive to publish minor releases relatively frequently. We generally try and focus on a three-month cadence for a minor release, starting at the first month of the calendar quarter and releasing a stable version of that minor release toward the end of the quarter.

For example, Orchestrator 2.0 entered pre-release review for testing at the start of January 2024, was released stable in March 2024. In addition, Orchestrator 2.1 entered development in early April and is planned for a stable release toward the end of June 2024.

What are the criteria for compatibility across versions?

This section is intended to be used by contributors to determine which changes are safe for a given release. This list is in no way exhaustive; it only outlines the most common situations seen during development.

The following are acceptable changes in patch releases:

- Fixing a bug in a way that has no major negative impact on most projects, such as a visual or physics bug. If fixing a bug has a negative impact that could jeopardize a lot of projects, it should be made optional (using a project setting).
- Adding a new optional parameter to a method.
- Small-scale editor usability tweaks.

Note that maintainers may be more conservative with fixes in subsequent patch releases. For instance, 2.0.1 may receive more impactful fixes than 2.0.2.

The following changes are acceptable in minor releases, but not patch releases:

- Significant new features.
- Renaming of method parameters due to integration limitations with C# and GDScript.
- Deprecating methods, variables, or classes. This is done by adding a deprecated flag to the class reference, which should show in the editor. When a method is marked deprecated, it's slated to be removed in the next *major* release.
- Changes that affect the default visuals.
- Bug fixes that significantly change the behavior or output, with the aim to meet user expectations better. In comparison, in patch releases, we favor keeping buggy behavior to avoid breaking existing projects who may rely on that wrong behavior or use workarounds.
- Performance optimizations that result in visual changes.

The following changes are considered **compatibility-breaking** and can only occur in new major releases:

- Renaming or removing methods, variables, or classes.
- Modifying a node's inheritance or behavior to where the node acts entirely different.
- Changing the default value of a setting in a way that affects existing projects.

As Orchestrator 3.0 hasn't been branched off yet, we strongly discourage any compatibility breaking changes.

System requirements

This page outlines the system requirements for the Orchestrator plug-in. These specifications are given for informative purposes only, but they can be referred to if you're looking to build or upgrade a system to use develop games using Orchestrator.

Orchestrator

Orchestrator is a plug-in that runs inside the Godot Editor during development. Therefore, the use of the plug-in depends solely on meeting the requirements to run the Godot Editor.

The current **minimum** specifications required to run the Godot Editor are available in the Godot documentation.

Exported Orchestrator project

Similarly, to use Orchestrator in an exported Godot game, the plug-in must run inside the Godot engine. Therefore, the use of the plug-in depends solely on meeting the requirements to run the Godot engine.

The current **minimum** specifications required to run an exported Godot project are available in the Godot documentation.

Tutorials and resources

This page maintains a list of various tutorials and resources created by Crater Crash Studios and community contributors.

If you believe something is missing here, submit a Pull Request.

Tutorials

- Godot Documentation 2D Tutorial with Orchestrator
- Godot Documentation 3D Tutorial with Orchestrator
- Step-By-Step Example Guide

Showcase Videos

- Godot Visual Script is Back! - Credit: @Gamesfromscratch, February 2024.
- I Tried Godot VisualScript In 2024 (And Found An Alternative) - Credit: @Code_It_All, April 2024.

Contribution workflow

Pull request workflow

The so-called “PR workflow” used by Orchestrator is common to many projects using Git, and should be familiar to veteran free software contributors. The idea is that only a small number (if any) commit directly to the master branch. Instead, contributors fork the project (i.e. create a copy of it, which they can modify as they wish), and then use the GitHub interface to request a pull from one of their fork’s branches to one branch of the original (often named upstream) repository.

The resulting *pull request* (PR) can then be reviewed by other contributors, which might approve it, reject it, or most often request that modifications be done. Once approved, the PR can then be merged by one of the core developers, and its commit(s) will become part of the target branch (usually the master branch).

We will go together through an example to show the typical workflow and associated Git commands. But first, let’s have a quick look at the Orchestrator Git repository.

Git source repository

The repository on GitHub is a Git code repository together with an embedded issue tracker and PR system.

If you are contributing to the documentation, its repository can be found [here](#).

The branches on the Git repository are organized as follows:

- The **main** branch is where development for the next major version occurs. As a development branch, it’s considered unstable and not meant for production use. This is where pull requests should **always** be done.
- The stable branches are named after their respective version, e.g. **1.1** and **2.0**. They are used to backport bugfixes and enhancements from the **main** branch to the currently maintained stable release. As a rule of thumb, the last stable branch is maintained until the next minor version (e.g. the **2.0** branch maintained until **2.1**). If you want to open pull requests against a stable branch, please check first if your changes are relevant for the **main** branch, and make the PR against **main** if so. The core team will cherry-pick from **main** if they believe the change should be included in a maintenance release for a stable branch.

Forking and cloning

The first step to contributing is to *fork* the repository on GitHub. To do so, you must have a GitHub account, be logged in, and click the button in the top right that reads “Fork”. You will need to answer a series of questions. Once

complete, you will find a repository under your username in the URL with the same upstream repository name of `godot-orchestrator`.

Once you have forked the repository, it's time to make a local copy, so you can more easily make changes. In Git speak, you must clone your fork to your local computer. To clone your repository, use the following command in a terminal window:

```
git clone https://github.com/<USERNAME>/godot-orchestrator.git
```

After Git has cloned the repository, you will find a new directory called `godot-orchestrator`. Next, change directories into the `godot-orchestrator` directory and execute the following commands:

```
git remote add upstream https://github.com/CraterCrash/godot-orchestrator.git
git fetch upstream
```

The above creates a reference named `upstream` to the original `godot-orchestrator` repository maintained by Crater Crash Studios. This reference will be helpful because you want to keep your local repository up-to-date with new commits added to the original repository. You will also have another reference to `origin`, which points to your *fork* repository on GitHub.

Rebasing changes on conflicts

Ideally, it would help if you created a feature or bugfix branch locally. This branch is where you will make all the necessary changes to add your new feature or bug fix.

When submitting pull requests for code changes, the commit history should be linear, not contain merge commits, and be free of merge conflicts. If you contribute a pull request and GitHub states there are conflicts, consider pulling down the latest upstream changes and rebasing your work on those changes before opening the pull request.

Assuming you have set up the `upstream` reference and you have committed all your changes in your local fork's branch, execute the following commands to rebase:

```
git checkout main
git pull upstream main
git checkout <your branch>
git rebase main
```

If there are conflicts, you must compare the upstream changes with your local changes in your feature branch. You can continue the rebase by running `git rebase --continue` after resolving all conflicts. Once the rebase has finished, you must update your feature branch using `push --force` to your GitHub fork's branch.

Code style guidelines

When contributing to Orchestrator's source code, you will be expected to follow the style guidelines outlined below. Some of them are checked via Continuous Integration and reviewers will ask that you fix potential issues, so it's generally a good idea to set up your system as outlined below to mitigate issues with your commits early.

C++ and Objective-C

There are no written guidelines, but the code's general style is agreed upon by the core developers, and is enforced via the `clang-format` code beautifier. This takes care of the following for you:

- Indentation and alignment are both tab based (respectively one and two tabs)
- One space around match and assignment operators, as well as after commas
- Pointer and reference operators are affixed to the type name, not the variable identifier
- See further down for header includes

The rules for the `clang-format` are outlined in the `.clang-format` file in the Orchestrator Git repository.

As long as your style matches the surrounding code and that you are not introducing trailing whitespaces or space-indentation, you should be fine. If you plan to contribute regularly, however, we strongly advise that you set up `clang-format` to check and fix all your commits.

Orchestrator's code style guidelines should not be applied to third-party or external git submodule code. In other words, it should only be applied to the files contained within the `/src` directory.

File names

Use `snake_case` for file names. Avoid using capital letters, as these are handled slightly differently across operating systems.

Header includes

When adding new C++ or Objective-C files, or including new headers in existing ones, the following rules should be followed:

- The first lines in the file should be Orchestrator's copyright header and Apache License 2.0, copy-pasted from another file.
- In a `.h` header, include guards should be used with the form `ORCHESTRATOR_FILENAME_H` or `OSSCRIPT_FILENAME_H`.

- In a `.cpp` file (e.g. `filename.cpp`), the first include should be the one where the class is declared (e.g. `#include "filename.h"`), followed by an empty line for separation.
- Then come headers from Orchestrator’s code base, included in alphabetical order, with paths relative to the root folder. The includes should be surrounded using quotes, e.g. `#include "core/object.h"`. The block of Orchestrator includes should then be followed by an empty line for separation.
- Then come headers from Godot’s `godot-cpp` library, included in alphabetical order. The includes should be surrounded with brackets, e.g. `#include <godot_cpp/classes/control.hpp>`, followed by an empty line for separation.
- Finally, any remaining includes should be included, again using the brackets, listed in alphabetical order.

```
text title="Example Header" // This file is part of the Godot
Orchestrator project. // // Copyright (c) 2023-present Crater
Crash Studios LLC and its contributors. // // Licensed under
the Apache License, Version 2.0 (the "License"); // you may not
use this file except in compliance with the License. // You may
obtain a copy of the License at // //      http://www.apache.org/licenses/LICENSE-2.0
// // Unless required by applicable law or agreed to in writing,
software // distributed under the License is distributed on
an "AS IS" BASIS, // WITHOUT WARRANTIES OR CONDITIONS OF ANY
KIND, either express or implied. // See the License for the
specific language governing permissions and // limitations under
the License. // #ifndef ORCHESTRATOR_ABOUT_DIALOG_H ... #endif
ORCHESTRATOR_ABOUT_DIALOG_H
```

```
“text title=“Example Implementation (cpp)” // This file is part of the Godot
Orchestrator project. // // Copyright (c) 2023-present Crater Crash Stu-
dios LLC and its contributors. // // Licensed under the Apache License,
Version 2.0 (the “License”); // you may not use this file except in compli-
ance with the License. // You may obtain a copy of the License at // //
http://www.apache.org/licenses/LICENSE-2.0 // // Unless required by appli-
cable law or agreed to in writing, software // distributed under the License is
distributed on an “AS IS” BASIS, // WITHOUT WARRANTIES OR CONDI-
TIONS OF ANY KIND, either express or implied. // See the License for the
specific language governing permissions and // limitations under the License. //
#include “about_dialog.h”
```

```
#include “authors.gen.h” #include “donors.gen.h” #include “license.gen.h” #in-
clude “common/version.h” #include “plugin/plugin.h”
```

```
#include <godot_cpp/classes/display_server.hpp> #include <godot_cpp/classes/editor_interface.hpp>
#include <godot_cpp/classes/font.hpp> #include <godot_cpp/classes/h_box_container.hpp>
#include <godot_cpp/classes/h_separator.hpp> #include <godot_cpp/classes/label.hpp>
#include <godot_cpp/classes/os.hpp> #include <godot_cpp/classes/scroll_container.hpp>
```

```
#include <godot_cpp/classes/style_box_empty.hpp> #include <godot_cpp/classes/style_box_flat.hpp>
#include <godot_cpp/classes/tab_container.hpp> #include <godot_cpp/classes/texture_rect.hpp>
#include <godot_cpp/classes/theme.hpp> #include <godot_cpp/classes/v_box_container.hpp>
```

Class names

All C++ class names should use ****Pascal Case****.

Additionally, there are two naming conventions that should be followed, depending on the context. Editor functionality related classes should be prefixed with ``Orchestrator``. Runtime script functionality related classes should be prefixed with ``OScript``.

* ****Editor Example****: ``OrchestratorMainView`` or ``OrchestratorPlugin``.
 * ****Script Example****: ``OScriptNodeConstant`` or ``OScriptNodeInitContext``.

As a rule of thumb, we prefer one class per file, but there are some instances where it may not be possible. Please do so sparsely, preferring one class per file ideally.

Function and variable names

All functions and variables should be defined using ``snake_case``.

If a function or variable is not public, it should be prefixed with an underscore (``_``).

Additionally, all function arguments should be prefixed with ``p_`` unless the value is being modified.

```
```cpp title="Example"
class MyClass : public Resource
{
 GDCLASS(MyClass, Resource)

protected:
 String _the_name;

 void _set_the_name(const String& p_name);

public:
 String get_the_name() const;
};
```

Additionally, function and variables of like scope should be grouped together. Lastly, all functions that do not override a parent class method should be documented. See the Comment style guide for more details.

## Constants and enums

Constants should use `CONSTANT_CASE` (also called screaming snake case), that is all caps with an underscore (`_`) to separate words.

```
cpp title="Example constant" const MAX_SPEED = 10;
```

Define enums using PascalCase, with a prefix of E:

```
cpp title="Example enum" enum EDataType { EDT_ANY, EDT_NUMERIC
};
```

## Comment style guide

This comment style guide applies to all languages within the Orchestrator code base.

- Begin comments with a space character to distinguish them from disabled code.
- Use sentence case for comments, beginning with an uppercase character and ending with a period.
- Reference variables/functions using backticks
- Wrap comments to ~120 characters.
- Use TODO:, FIXME:, NOTE:, or HACK: as admonitions as needed.

```
cpp title="Example" // Compute the first 10,000 decimals of Pi.
// FIXME: Don't crash when computing due to `increment` being
negative.
```

Don't repeat what the code says in comments, explain *why* rather than *how*.

```
cpp title="Poor example" // Draws loading screen. draw_loading_screen();
```

Ideally all methods should be documented, using doxygen style comments.

```
cpp title="C++ method documentation example" /// Returns the
number of nodes in the universe. /// @param p_root the root node,
should not be <code>null</code> /// @return the number of nodes
uint64_t Universe::get_nodes_from_root(Node* p_root) { // ... }
```

For member variables, use single-line comments at the end of the variable declarations:

```
cpp title="C++ member variable documentation" public: boolean
_is_visible; //! Specifies whether this object is visible
```

## C++ usage guidelines

### Rationale

Since Orchestrator 2.0, Orchestrator was written in C++ and the standard used throughout the code base is **C++20**. While modern C++ brings a lot of opportunities to write faster, more readable code, we chose to restrict the usage for a number of reasons:

- It makes it easier to review code in online editors. This is because contributors don't always have access to a full-featured IDE while reviewing code.
- It makes the code easier to grasp for beginner contributors who may not be as proficient in C++.

For your pull request to be merged, it must follow the C++ usage guidelines outlined here. If you are extending Orchestrator in your own fork, you can use features not allowed in your own code, but any contributions upstream should follow these rules.

## Disallowed features

**Any feature that isn't listed is allowed.**

Using features like `constexpr` variables and `nullptr` is encouraged when possible. Still, try to keep your use of modern C++ features conservative. Their use needs to serve a real purpose, such as improving the code's readability or performance.

## Standard Template Library

We do not allow the use of the STL as Orchestrator utilizes all of Godot's own data types.

This means that pull requests should **not** use `std::string`, `std::vector`, and the like. Instead, use the Godot data types described below:

- Use `String` rather than `std::string`.
- Use `Vector` rather than `std::vector`. In some cases `LocalVector` can be an alternative.
- Use `Array` rather than `std::array`.

There are some Godot CPP data structures that do use `std::vector`, such as `MethodInfo`. For these specific corner cases, the use of `std::vector` is allowed, but it should be minimal.

## auto keyword

Please do not use the `auto` keyword for type inference. While it can avoid repetition, it can also lead to confusing code:

```
“cpp title=“Example” // Not so confusing... auto button = memnew(Button);
// ...but what about this? auto result = EditorNode::get_singleton()-
>get_complex_result();
```

Keep in mind that intellisense and hover documentation isn't readily available for pull requests. Most of the time, reviewers will use GitHub's online viewer to review pull requests.

We chose to forbid ``auto`` instead of allowing it on a case-by-case basis to avoid having to

### ### Lambdas

Lambdas should be used conservatively when they make code effectively faster or simpler, and Please ask before using lambdas in a pull request.

### ### ``#pragma once`` directives

The code now prefers the use of ``#pragma once`` directives over the C-style ``#ifdef``-based in Pull requests that do not use ``#pragma once`` directives will not be merged until changed.

```

sidebar_position: 3
title: Plugin development

```

### # Plug-in development

The guides below explain how to contribute to the plug-in's code base.  
You will learn the best practices when working on solutions and features, the source code st

- \* [Code style guidelines](code-style-guidelines)
- \* [C++ usage guidelines](cpp-usage-guidelines)

```

sidebar_position: 1

```

### # Ways to contribute

Orchestrator is a community-driven, free, and open-source project.  
All developers work on this plug-in *\*pro bono\**, meaning they contribute their free time and

This means that for Orchestrator to thrive, it needs contributors to help improve the plug-in  
There are multiple ways to contribute to a project, making it possible for everyone to bring

- **Be part of the community**.<br/>  
The best way to contribute to Orchestrator, and to help it grow is to simply use the plug-in  
Participate! Being a user and advocating for the plug-in helps spread the word about Orchest

- **Make games**.<br/>  
It's no secret, no convince new users that visual scripting with Orchestrator can be fun, we  
We know that visual scripting has tremendous potential, it's one of the most common ways to  
So work on your awesome projects, each new game increases the plug-ins ability to illustrate

- **\*\*Get involved in the plug-in's development\*\***.<br/>

Many people think that getting involved in develop means contributing code, but that's far from the only way. While contributing code via pull requests are much appreciated, testing the development snapshots is also very helpful. The following sections will cover some of these direct ways to contribute.

- **\*\*Donate\*\***.<br/>

Orchestrator is a plug-in that is defined completely free of charge, and in the spare time of its maintainers. Donations help cover certain operating costs for hosting our websites, content, and more. If you want to donate to the project, check [ExternalLink href="https://orchestrator.cratercrash.org/donate"](https://orchestrator.cratercrash.org/donate)

## ## Contributing code

The possibility to study, use, and contribute to the Orchestrator project is made possible by its open source nature.

If you want to get started contributing to open source, check out the repository by creating an account on GitHub. One of the most direct ways to contribute to the project is to submit bug fixes or improvements.

Contributing code has two big advantages:

- \* Your own code will be reviewed and improved by other developers, and will be further maintained. On the other hand, it comes with a responsibility, as your changes need to be generic to be beneficial to the community.

- \* The whole community benefits from your work, consequentially other contributors will continue to improve the project.

To aid in collaboration and overall quality, the core developers will enforce some rules for contributions.

All pull requests go through a review process before being accepted.

Depending on the scope of the changes, it may take sometime for a maintainer to provide the feedback. We value all contributors, and ask for patience when submitting pull requests.

## ## Testing and reporting issues

Another excellent way to contribute to the plug-in is to test development releases and report issues. It is also helpful to report issues discovered during stable releases, so that they can be fixed in the next version.

### ### Testing development versions

To help with testing, test our official pre-release binaries when they are announced (using `godot-orchestrator-rc`). Pre-release builds are tagged with the ``dev``, ``beta``, or ``rc`` designators.

It's also helpful to be on the lookout for bugs in stable releases.

### ### Filing an issue on GitHub

Orchestrator uses [ExternalLink href="https://github.com/CraterCrash/godot-orchestrator/issues"](https://github.com/CraterCrash/godot-orchestrator/issues) for issue tracking. When you start filing a bug report, you'll be given a form to fill out.

Please try and follow it so that all issues are consistent and provide the necessary information.

## ## Contributing to the documentation

The documentation is maintained in the `<ExternalLink href="https://github.com/CraterCrash/godot-editor-orchestrator">`. This repository maintains not only the raw markdown files used for documentation, but also a

Should you find outdated information in the documentation or parts that could be improved, c

---

sidebar\_position: 4

---

## # Writing documentation

We always can use help to improve documentation, be it the script node reference or the manu

---

sidebar\_position: 4

---

## # First look at the plug-in

This page will give you a brief overview of Orchestrator's interface.  
We're going to look at the main workspace, the integration with Godot, and its intuitive int

## ## The main workspace

You access the main workspace of the plug-in by clicking on the tab labeled **Orchestrator**.

`<Figure image="/img/common/godot-editor-orchestrator-tab.png" caption="Orchestrator main tab"`

## ### Graph view

Initially, the main workspace is empty, only showing the file list.  
This is because no ``Orchestration`` resources have been opened.  
By either creating a new orchestration using ``File > New Orchestration``, or opening an exist

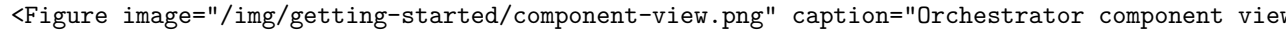
`<Figure image="/img/getting-started/graph-workspace.png" caption="Orchestrator graph workspace"`

In the above example, an orchestration has been opened.  
To the left is the **Graph** canvas where you place visual script nodes to create logic.  
At the top of the **Graph**, there is a tab bar, where you can quickly navigate between dif

On the right is the **Component** view, where high-level components can be added.

### ### Component view

The **Component** view is where you can quickly add, rename, and remove various components from your graph.

The image shows the Orchestrator component view, which is a workspace for adding and managing components in a graph.

- \* The **Graph** section always defines at least one graph called **EventGraph**. This is where you will define all your interactions with the Godot engine to handle overridden events.

- \* The **Functions** section allows for defining user-defined functions. User-defined functions are a great way to not only expose functionality to the outside of your script but also to reuse code within your script.

- \* The **Macros** section is currently unimplemented, and will be enabled in a future patch.

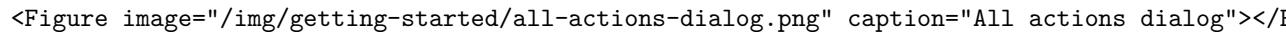
- \* The **Variables** section allows creating unique named objects for storing values. Variables are associated with a specific Godot type and can be publicly accessible or scoped. A variable will be publicly accessible if it has the `<EditorIcon name="GuiVisibilityVisible"/>` icon. This means that the variable is exported, in Godot terms. The variable is not publicly accessible when using the `<EditorIcon name="GuiVisibilityHidden"/>` icon. This means that the variable is not exported, in Godot terms.

- \* The **Signals** section is for user-defined signals, allowing outside observers to connect to them.

To add a new **Component** type to the ``Orchestration``, use the `<EditorIcon name="Add"/>` button. Additionally, for **Functions**, you can also override several Godot built-in virtual functions.

### ### All actions dialog

To begin adding logic to your script, use the right-mouse button to open the **All Actions** dialog.

The image shows the All actions dialog, which is a search interface for finding and adding actions to the graph.

This dialog is where you can search for specific nodes that you wish to add to the graph. To add a node, either select the choice and press the **Add** button or simply press **Enter**.

Actions are grouped into the following high-level categories.

:::info

The `<EditorIcon name="Favorites"/>` icon can be used to add any number of actions as a favorite. If you use a specific action frequently, it's recommended that you click the `<EditorIcon name="Favorites"/>` icon. Favorites are shown in the top-left of the **All Actions** dialog. If you later wish to unfavorite an action, simply click the favorite item in the list and press the **Unfavorite** button.

In addition to favorites, your most recently chosen items are listed in the bottom-right of the dialog.

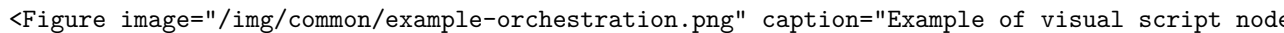
:::

:::tip

Right-clicking the graph canvas opens the **All Actions** dialog using the script's base type.  
If you drag from a visual script node's pin and release the mouse, the **All Actions** dialog also opens.  
:::

### Visual script nodes

All orchestration scripts are comprised of various visual script nodes.  
A visual script node is the building block of an orchestration, that you connect with other nodes.

Example of visual script node

Script nodes have two unique pin types:

- \* `<EditorIcon name="VisualShaderPort"/>` Execution or control flow pins that define the order of execution.
- \* `<EditorIcon name="GuiGraphNodePort"/>` Data pins that define input or output data values, color, etc.

The pins that are on the left-side of the node are called **Inputs**.  
The pins on the right-side of the node are called **Outputs**.

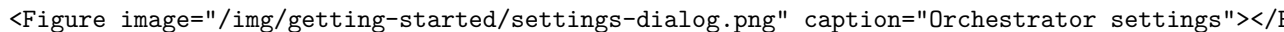
Visual script nodes can be added to an orchestration using various methods, which include:

- Opening the **All Actions** dialog
- Or by dragging from various editor docks and the **Component Panel**.

For details on all the script nodes available, please see the [Script Node Reference](../Script Node Reference/Script Node Reference).

## Customization and configuration

Orchestrator also provides a number of customizations.  
These can be accessed from the **Orchestrator** main tab and navigating to `File > Settings`.

Orchestrator settings

In the **General** tab, all plugin configuration settings can be adjusted, including behavior and preferences.

Additionally in the `Interface > Theme` section, you can customize the titlebar colors and other interface elements.

In the **Shortcuts** tab, plugin shortcuts can be customized to your needs.

## Seamless editor integration

One of the major goals the Orchestrator development team has is to make sure that user experience is seamless.  
Users should feel as though the plug-in is a fundamental, core part of the Editor, not some add-on.

To achieve this goal, not only do we do our best to follow the same design principals of the Editor, but we also ensure that the plug-in is tightly integrated with the Editor's workflow.

### ### Drag-n-drop anywhere

One of the major benefits of Orchestrator is it recognizes data elements that are from other

The following drag-n-drop options exist:

- Drag a scene node from the **Scene** view to get a reference to that scene node in the script
- Drag a property from the **Inspector** view to access either a **Get** or a **Set** for the property
- Drag a resource from the **FileSystem** view to either obtain its path or to create a **Path** property

### ### Multi-node operations

Bulk operations improves efficiency, and having access to such tools makes working with visual scripting easier. You can use the **Left Mouse Button** to click on the graph and begin to drag to select multiple nodes.

With multiple node selected, you can `<EditorIcon name="ActionCut"/> cut (`Ctrl+X`)`, `<EditorIcon name="ActionPaste"/> paste (`Ctrl+V`)`, or `<EditorIcon name="ActionCopy"/> copy (`Ctrl+C`)`.

### ### Undo and redo

At this time, Orchestrator does not officially support Undo/Redo operations in the Godot Editor.

:::info

This feature is currently under development and is planned for a future release.

:::

## ## File formats

Orchestrator supports two file formats:

- \* `.torch`, which is text-based
- \* `.os` which is binary-based.

The text-based format was introduced with Orchestrator 2.1, and is the default.

This format is more human-readable and is easier to work with version control systems, as text files are easier to diff.

The binary-based format was originally introduced with Orchestrator 2.0, and is planned to be replaced by the text-based format in a future release.

### ### Converting between formats

In order to convert from one format to another, simply follow these steps:

1. Open the desired orchestration. This can be performed by double-clicking the file in the **FileSystem Dock** by using the **Open** button.
2. Click on the **File** menu and select **Save As**.
3. In the **Save As** dialog, provide a new file name with the desired format extension (`.torch` or `.os`).

```
:::warning
When converting an Orchestration between formats, the scenes that import and use the Orchestration
You will need to open each scene that uses the Orchestration and re-attach the new Orchestration
:::
```

```
:::tip
If you want to change the default format used by the plug-in, this can be done in the `File` menu.
Simply open the dialog and navigate to the `Editor > Settings` section where you will find the
Here you can select between either Text or Binary, which controls the default format.
:::
```

### ### Exporting games

In a Godot project, not every resource is exported in the same format that it's saved with in the project. This is because often times resources are saved using human-readable formats in the project. Several built-in Godot resources are converted to binary when exporting games, such as `.tres` files.

Orchestrator also uses this same technique when the project's Orchestration is stored in text format. During the export process, the `.torch` file is converted to a compressed, binary format, which is then included in the game's export package.

```
:::tip
When exporting a game, any .os binary format that is used in the project is exported as-is.
This is because the binary format is already in a compressed format that is suitable for fast loading.
:::
```

### ## Integrated help

You can access the documentation related to any visual script node by selecting the node, right-clicking it, and choosing `View Documentation`. This automatically redirects the Editor's main view to the Editor Help window, where the script's documentation is displayed.

```

sidebar_position: 1

```

### # Introduction

This series will introduce you to Orchestrator and give you an overview of its features.

In the following pages, you will get answers to questions such as "Is Orchestrator right for my project?" We will introduce the plug-in's most essential concepts, showcase the in-editor interface, and provide a detailed guide to its usage.

- \* [Introduction to Orchestrator](introduction-to-orchestrator)
- \* [Overview of key concepts](key\_concepts)
- \* [Installation Guide](installation-guide)

- \* [First look at the plug-in](first-look-at-the-plugin)
- \* [Learning new features](learning-new-features)
- \* [Orchestrator's design philosophy](orchestrator-design-philosophy)

```

sidebar_position: 3

```

## # Installation guide

There two ways to install the plug-in:

- \* Using the Godot Asset Store <EditorIcon name="AssetStore"/>, formerly the Asset Library, [link](#)
- \* Downloading the plug-in from our <ExternalLink href="https://github.com/CraterCrash/godot-orchestrator">GitHub repository</ExternalLink>
- \* Downloading the plug-in from the <ExternalLink href="https://store.godotengine.org/asset/10000">Godot Asset Store</ExternalLink>

In this section, we'll cover all of these.

### ## Godot asset store in the editor

The Godot Editor has a built-in asset store feature, which allows users to search, download, and install assets.

To access the **Asset Store**, we'll use the <EditorIcon name="AssetLib"/> **Asset Store** button in the top right corner of the editor.

<Figure image="/img/common/godot-editor-assetlib.png" caption="Godot Asset Store"></Figure>

In the search box, type ``Orchestrator`` or simply ``Scripting`` to locate the plug-in.

<Figure image="/img/common/godot-editor-assetlib-search.png" caption="Search for Scripting /"></Figure>

Click the plug-in's name to open the **Information** dialog.

<Figure image="/img/common/godot-editor-assetlib-plugin.png" caption="Orchestrator plug-in information dialog"></Figure>

Press the **Download** button to begin the download process.

When the download finishes, you'll be prompted to **Configure Asset Before Installing**, shown below.

<Figure image="/img/common/godot-editor-assetlib-configure.png" caption="Configure plug-in dialog"></Figure>

Uncheck the ``Ignore asset root`` option.

<Figure image="/img/common/godot-editor-assetlib-ignore-asset-root.png" caption="Uncheck ignore asset root option"></Figure>  
:::warning

Failing the uncheck this option will not install the plug-in correctly into your project.  
:::

Finally, click the **Install** button, installing the plug-in into the ``res://addons/orchestrator``. After installation, the Godot Editor will prompt you to restart.

:::tip

There is currently an issue with the Godot Editor with loading textures used by GDEXTENSION. The plug-in will be loaded before the textures are imported, causing the textures to not render. Restarting the Godot Editor one final time fixes the issue.

:::

## ## Download from GitHub

If you are interested in pre-release builds or are having trouble installing the plug-in using the Godot Asset Store, To install from GitHub, simply follow these steps:

1. Locate the latest release on the repository's `<ExternalLink href="https://github.com/Craftworkbench/godot-orchestrator">GitHub repository</ExternalLink>`.  
:::note  
While the repository page will show the latest **stable** release, if you are interested in pre-release builds, click the **Releases** tab.  
:::  
2. Once you've located the desired release, expand the **Assets** section if it isn't already expanded.  
`<Figure image="/img/common/github-release-assets.png" caption="GitHub release assets"></Figure>`  
3. Locate the asset labeled ``godot-orchestrator-<version>-plugin.zip``, and click it.  
This will download the artifact to your PC.

Once the artifact has been downloaded, it needs to be unzipped into your Godot project.

If you have not already created a Godot project, you should create one first following `<ExternalLink href="https://docs.godotengine.org/en/stable/getting_started/first_project.html">this guide</ExternalLink>`.

With the Godot project created, navigate to where you saved the downloaded file, and extract it. This should create a directory path like ``<project-root>\addons\orchestrator``.

Once the files have been installed, focusing the Godot Editor will request to restart.

Simply follow the prompt, restarting Godot, and the plug-in should be active when the editor starts.

:::tip

There is currently an issue with the Godot Editor with loading textures used by GDEXTENSION. The plug-in will be loaded before the textures are imported, causing the textures to not render. Restarting the Godot Editor one final time fixes the issue.

:::

## ## Download from the Godot Asset Store

If you are having trouble installing the plug-in using the Godot Asset Store in the Editor, To install from Asset Store, simply follow these steps:

1. Locate the version you are interested in on the right, the big **Download** button. There is a drop-down version if you need to download an older release. This will download the artifact to your PC.

Once the artifact has been downloaded, it needs to be unzipped into your Godot project.  
If you have not already created a Godot project, you should create one first following <Ext

With the Godot project created, navigate to where you saved the downloaded file, and extract  
This should create a directory path like ``<project-root>\addons\orchestrator``.

Once the files have been installed, focusing the Godot Editor will request to restart.  
Simply follow the prompt, restarting Godot, and the plug-in should be active when the editor

`:::tip`  
There is currently an issue with the Godot Editor with loading textures used by GDExtension  
The plug-in will be loaded before the textures are imported, causing the textures to not render  
Restarting the Godot Editor one final time fixes the issue.  
`:::`

`---`  
`sidebar_position: 1`  
`---`

## `# Introduction to Orchestrator`

This article is to help you determine if Orchestrator might be a good fit for you.  
We will introduce some broad features of the plug-in and give you a feel for what is achievable

This is by no means an exhaustive overview.  
We will introduce many more features in this getting started series.

## `## What is Orchestrator?`

Orchestrator is a Godot plug-in designed to provide visual-scripting capabilities to any Godot  
It can be used to create games or applications that are released on a myriad of platforms, i

You can also create console games using Orchestrator, although you either need strong program

`:::note`  
The Orchestrator development team cannot provide an open source console export due to licens  
Regardless of the engine used, releasing games on consoles is always a lot more work than o  
You can read more in the <ExternalLink href="https://docs.godotengine.org/en/stable/tutorial  
`:::`

## `## What can the plug-in do?`

The idea of Orchestrator as a visual-scripting tool began as an in-house experiment for sever  
The team soon began to see the benefit of open sourcing the project for other Godot develop

The plug-in is designed to fully integrate with the Godot engine, allowing developers to vis

Here is a very simple example of an `Orchestration`, the main visual script resource.  
This script is designed to rotate an object at constant speed each frame tick:

<Figure image="/img/common/example-orchestration.png" caption="An example orchestration"></F

You can find many more examples in our <ExternalLink href="https://github.com/CraterCrash/g

## How to install the plug-in?

Orchestrator does not ship with the Godot Editor, and instead it must be installed into your  
For information on how to install the plug-in into a new project, please read the [Installat

## How does it work and look?

Orchestrator provides you with an entirely new workspace tab in the main editor viewport to

<Figure image="/img/common/godot-editor-orchestrator.png" caption="Orchestrator main view">

The team strives to not only provide a feature-rich experience with the power of visual scri  
Any project is also evolving, so while we believe the integration is done quite well, there

## What language is an Orchestration?

The Godot engine provides two core programming languages out of the box, `GDScript` and `C#`  
These two languages provide seamless, powerful scripting opportunities in the engine; howeve

Orchestrator provides a new type of programming language to Godot, to create new script res  
An orchestration, also called an `OrchestratorScript` or `OScript` for short, is a visual-sc

Orchestrator's code base is written in C++, the most popular language used in the gaming ind  
Developers use C++ because it's highly optimized and efficient, which is critical in high-pe

However, as a developer using Orchestrator, you aren't required to know C++, or really any t  
The plug-in is designed to connect visual blocks of functionality together to create logic.  
An `Orchestration` is translated automatically to machine instructions at runtime, so that t

And finally, the plug-in seamlessly integrates with the Godot Editor and Godot Engine thanks

## What do I need to know to use Orchestrator?

Orchestrator is a feature packed plug-in for Godot, with thousands of features.  
Its goal is to abstract away a lot of the technical noise around syntax and specific languag

While we try and make the plug-in accessible to the most junior of developers, having a basi

As we continue to refine the plug-in and find more ways to simplifying its design, your input is appreciated.

In the next part, you will get an overview of the plug-ins essential concepts.

---

sidebar\_position: 2

---

## # Overview of key concepts

Orchestrator is a Godot-plugin that is based on a set of key abstractions.

An ``Orchestration`` is the main resource that you will be creating and managing using the Orchestrator. It represents a Godot ``Script`` object that contains one or more graphs with any number of connections.

Orchestrations also support user-defined functions, variables, and signals, much like you would in a Godot ``Script``. Since an orchestration is considered a Godot ``Script``, you can attach it to any scene node, and it will be executed when the scene is loaded.

There are **6** major concepts, and we're going to look at them briefly to give you a sense of what they are.

### ### Scenes

While scenes are not something specific to Orchestrator, they are the foundation to which Orchestrator is built. It's extremely important to understand what scenes are, how they're used, and why they're important.

In Godot, developers break down a game into reusable components, called *\*scenes\**.

A scene can literally be anything, a character, a weapon, a menu in the user interface, a simple object, etc. Godot's scene system is designed to be flexible; providing the role of both prefabs (archetypes) and objects.

<Figure image="/img/getting-started/key\_concepts\_main\_menu.webp" caption="Simple scene"></Figure>

For more information about Scenes, see the [Godot documentation] ([https://docs.godotengine.org/en/stable/tutorials/scene/scene\\_intro.html](https://docs.godotengine.org/en/stable/tutorials/scene/scene_intro.html)).

### ### Nodes

A scene is composed of one or more *\*nodes\**.

Nodes are Godot's smallest building blocks that you arrange into tree hierarchies.

<Figure image="/img/getting-started/key\_concepts\_character\_nodes.webp" caption="Simple node hierarchy"></Figure>

In this example, it is made up of a ``CharacterBody2D`` node named "Player", a ``Camera2D``, a ``Sprite2D``, and a ``Text`` node. The ``CharacterBody2D`` or "Player", is the root of this scene.

The other nodes are children of the root scene node, which creates a set of node hierarchies.

:::info

The node names end with `2D` because this is a two-dimensional scene; however 3D counterparts exist as well.

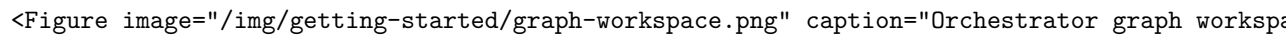
If you look closely at the node in the scene called "Player", there is an `<EditorIcon name="Script"/>` icon. This icon signifies that the node has a script attached.

The visual scripts you create with Orchestrator, called `Orchestration`s, are scripts that are attached to nodes. When they are attached, the `<EditorIcon name="Script"/>` is shown in the scene view next to the node.

For more information about Nodes, see the [Godot documentation](https://docs.godotengine.org/en/stable/classes/class\_@nodes.html).

### ### Graph workspace

The **Graph** is the main workspace where you will spend the vast majority of your time creating visual scripts. It is the canvas where you will place nodes, connect them together to create complex behaviors.

A screenshot of the Godot Orchestrator graph workspace, showing a canvas with various nodes and connections.

To the right of the **Graph** canvas is what is called the **Component** view. The **Component** view is where you can add, rename, and remove a variety of high-level components.

An `Orchestration` is not restricted to just a single **Graph** workspace. You can add multiple graphs using the `<EditorIcon name="Add"/>` button at the top of the **Graph** workspace.

In addition, when you add a user-defined function, these will also have their own **Graph** workspace. In the image above, you can see several tabs for **EventGraph**, **apply\_rotation**, and other functions. This technique allows for optimal organization in visual scripts, preventing clutter and separating concerns.

The **Component** view also allows for creating **Variables** and **Signals**.

### ### Signals

All Godot objects can emit signals, a notification to an observer when a specific event occurs.

```
:::info
Signals are Godot's version of the *observer* pattern.
:::
```

Signals allow you to communicate with other parts of the game without creating a hard dependency. This allows for a significant amount of flexibility when building scenes, and how you can connect them.

To add a signal to the `Orchestration`, use the **Component** view and click the `<EditorIcon name="Signal"/>` button.

Observers can be connected to signals either through code or using the Editor's interface. The way to connect a signal to an `Orchestration` is using the Editor's interface.

<Figure image="/img/getting-started/key\_concepts\_signals.webp" caption="Signals"></Figure>

Locate the **Node** view, this is in the same dock where you will find the **Inspector** and **Console**.  
In the **Node** view, there are two distinct subsections, `_Signals_` and `_Groups_`.  
Make sure that the `_Signals_` section is highlighted, as shown above.

In this view, signals are represented with the `<EditorIcon name="Signal"/>` icon next to its name.  
If a signal has any connections, there will be child items listed under the signal, denoted by a plus icon.

For example, the above image shows that the function `_on_area_2d_body_entered()` will be called when a body enters the `Area2D`.  
This is an example of creating an `Area2D`, a 2D area trigger, and notifying some observer when it is entered.

For more information about signals, see the [\[Signals\]\(../../nodes/signals\)](#) node reference.

### ### Variables

A **Variable** represents a named value that can be assigned a value or its value read in the script.  
Variables are useful for passing data between different parts of the script, and for maintaining state.

To add a new variable, use the **Component** view and click the `<EditorIcon name="Add"/>` icon.

For more information about variables, see the [\[Variables\]\(../../nodes/variables\)](#) node reference.

```

sidebar_position: 5

```

### # Learning new features

Orchestrator is a tremendously feature-rich plug-in, directly driven by Godot's extensive library of features.  
There is most certainly a lot to learn.  
This page explains how you can use the online documentation, built-in code reference, and other resources.

### ## Making the most of the manual

What you are reading now is called the user manual.  
It documents the plug-in's concepts and features.  
When learning a new topic, you can start by browsing the corresponding section of this website.  
The left menu allows you to explore broad topics, while the right menu provides more targeted links.

The user manual has a companion [\[Script Node Reference\]\(../../nodes/all\\_nodes\)](#) that explains the details of the script language.  
While the manual covers general features, concepts, and how to use the plug-in, the reference covers the specifics of the script language.

**tip**  
At this time, the [\[Script Node Reference\]\(../../nodes/all\\_nodes\)](#) is only available online.

:::

## ## Thinking like a programmer

Teaching programming foundations and how to think like a game designer is beyond the scope of this manual. If you are new to programming, we recommend this free online course to get started:

- Harvard University offers free courses to learn to program, [`<ExternalLink href="https://cs50.org/`](https://cs50.org/). It teaches programming fundamentals and how to think like a programmer. Even when using visual tools, having this basic knowledge can easily improve your efficiency.

## ## Learn with the community

Orchestrator has a growing community of users. If you are stuck on a program or need help to understand how to achieve something, you can ask for help on our discord.

The best place to ask questions and find already answered ones is on our official discord.

## ## Tutorials and examples

This manual aims to provide a comprehensive reference to Orchestrator's features. Aside from the 2D and 3D getting started series, it does not contain tutorials to implement specific features. If you are looking for a tutorial about creating a role-playing game, platformer, or other, you can find them on our discord.

---

sidebar\_position: 6

---

## # Orchestrator's design philosophy

Now that you've gotten your feet wet, let's talk about Orchestrator's design.

:::important

Every visual scripting engine is different and fits different needs.

:::

Not only do various visual scripting systems offer a range of features, how those features are implemented also varies. Orchestrator attempts to take some of the best practices and concepts from other visual scripting engines.

## ## All-inclusive

Orchestrator aims to provide answers to most questions. We want game developers to be able to use full, visual tools to build game logic rather than being limited to a specific set of features.

We acknowledge that visual scripting is not for everyone, and we expect that respect to be reflected in the way we design the tool.

It's equally important to understand that there are those who simply struggle to learn text.

Our mission and prime directive is to provide state-of-the-art tooling, and let you decide if

Games do not have to be made solely with or without one scripting technology like Orchestrator. You can safely mix-n-match, using Orchestrator for certain game aspects and combine GDScript with other technologies.

## ## Open source

Orchestrator is a fully, open-source code base under the **Apache License 2.0**. Anyone can freely use Orchestrator in any type of project, public, commercial, and proprietary.

See the [Complying with licenses](../about/complying-with-licenses) section for details.

## ## Community-driven

**Orchestrator is made by its community, for the community, and for all game creators.** The needs of users and open discussions are what drive the core updates we make to the plugin. New features from the core developers often focus on what will benefit most users first, and then on what will benefit the community.

---

sidebar\_position: 2  
toc\_max\_heading\_level: 4  
---

## # Step by step

This series builds upon the [Introduction to Orchestrator](../introduction) and will get you up to speed. If you have not yet reviewed the [Godot Step-by-Step Guide](https://docs.godotengine.org/en/stable/tutorials/editor/step-by-step-guide.html), we recommend you do so first.

- \* [Creating your first orchestration](#creating-your-first-orchestration)
- \* [Listening for player input](#listening-for-player-input)

## ## Creating your first orchestration {#creating-your-first-orchestration}

In this lesson, you will code your first `Orchestration` to make the Godot icon turn in circles. As we mentioned [in the introduction](../introduction), we assume you have read all the previous lessons.

<Figure image="/img/step-by-step/scripting\_first\_script\_rotating\_godot.webp" caption="Example of the Godot icon rotating in circles." data-bbox="213 735 1000 750"/>

### ### Project setup

Please <ExternalLink href="https://docs.godotengine.org/en/stable/tutorials/editor/project-setup.html" data-bbox="213 795 1000 810"/>. Your project should contain one picture: the Godot icon, which we often use for prototyping.

### #### Install the plug-in

Before we can proceed, we need to install the Orchestrator plug-in.  
The plug-in can be installed from the Godot Asset Library or manually by downloading from [our GitHub](#).

See the [\[Installation Guide\]\(../introduction/installation-guide\)](#) on how to install the plug-in.

#### #### Creating the scene

We need to create a ``Sprite2D`` node to display the Godot icon in our game.  
In the **Scene** view, click the **Other Node** button.

<Figure image="/img/step-by-step/scripting\_first\_script\_click\_other\_node.webp" caption="Click Other Node button" data-bbox="212 324 1000 340"/>

Type "Sprite2D" into the search box to filter the nodes, double-clicking on ``Sprite2D`` to create the node.

<Figure image="/img/step-by-step/scripting\_first\_script\_add\_sprite\_node.webp" caption="Add Sprite2D node" data-bbox="212 385 1000 400"/>

Once the `Sprite2D` node is added, your **Scene** view should now look like this:

<Figure image="/img/step-by-step/scripting\_first\_script\_scene\_tree.webp" caption="Scene view with Sprite2D node" data-bbox="212 444 1000 460"/>

A ``Sprite2D`` node requires a texture to display.

In the **Inspector** view on the right, the **Texture** property shows that it is *empty*.  
To display the Godot icon, drag the file ``icon.svg`` from the **FileSystem** view onto the **Texture** property.

<Figure image="/img/step-by-step/scripting\_first\_script\_setting\_texture.webp" caption="Set texture to icon.svg" data-bbox="212 551 1000 566"/>

:::important

You can create ``Sprite2D`` nodes automatically by dragging and dropping images onto the viewport.  
:::

Lastly, click and drag the icon in the viewport to center it in the game view.

<Figure image="/img/step-by-step/scripting\_first\_script\_centering\_sprite.webp" caption="Center the icon in the game view" data-bbox="212 671 1000 686"/>

#### ### Creating a new orchestration

To create and attach a new ``Orchestration`` to our node, right-click on the ``Sprite2D`` in the **Scene** view.

<Figure image="/img/step-by-step/scripting\_first\_script\_attach\_script.webp" caption="Right-click on Sprite2D node" data-bbox="212 761 1000 776"/>

The **Attach Node Script** dialog will appear.

This dialog allows you to select the script's language, the file path, and other options.  
Make sure the language is set to ``OScript``, leaving all the other options as their defaults.

Click the **Create** button to create the script.

<Figure image="/img/step-by-step/create-orchestration-dialog.png" caption="Attach node scrip

The Orchestrator **<EditorIcon name="ClassList"/> EventGraph** workspace should appear with

<Figure image="/img/step-by-step/create-orchestration-editor.png" caption="Editor window"></

```
Hello world!
```

Our ``Orchestration`` script currently doesn't do anything.

Let's make it print the text "Hello, world!" to the output bottom panel to get started.

1. Right-click the graph to open the **All Actions** dialog.
2. Search for ``Print String``, and select the ``Print String`` action.
3. To add the node, either press the **Add** button or simply hit **Enter**.
4. Change the value of the ``Text`` parameter to ``Hello, world!``.

Lastly, we need to add a Godot event callback so that our **Print String** node outputs the

1. Press the **<EditorIcon name="Override"/>** button located at the top right of the **Function**
2. Search for the ``Init`` action, found in the **Methods > Objects** category.
3. To add the node, either press the **Add** button or simply hit **Enter**.

With the two nodes in the **Graph**, using your mouse:

1. Left-mouse click the white *\*output\** arrow on the right side of the **Call Init** node.
2. Drag from this output pin to the *\*input\** arrow on the left side of the **Print String** node.

If done successfully, the two nodes should be connected.

:::note

The **Init** event is a special callback that represents the ``_init()`` function in GDScript. The Godot Engine calls this ``_init()`` on every object or node when it gets created, as long as it is in the scene.

:::

Now, save the scene as ``script_2d.tscn`` if you have not already, then press ``F6`` (``Cmd+R`` on Mac). Look at the **Output** panel in the editor, it should display "Hello, world!".

<Figure image="/img/step-by-step/scripting\_first\_script\_print\_hello\_world.webp" caption="Hel

Lastly, we need to remove this "Hello, world" logic, so:

1. Click on the **Print String** node, and right-click, selecting the **Delete** option to remove it.
2. Click on the **\_init** entry in the **Graphs** section of the **Component Panel**.

3. Right-click and select the **Remove** option from the context-menu.

You should end up with an empty canvas.

### ### Turning around

It's time to make our node move and rotate.

To do so, we're going to add two member variables to our **Orchestration**:

- \* The movement speed in pixels per second
- \* The angular speed in radians per second

<Figure image="/img/step-by-step/adding-variables.png" caption="Variables to add"></Figure>

### #### Add movement speed

To define the movement speed variable:

1. Click the <EditorIcon name="Add"/> button to add a new variable in the **Variables** comp
2. Give the variable the name ``speed``, and press enter.
3. Select the variable in the **Component** view to have the **Inspector** display the varia
4. In the **Inspector**, change the ``Type`` to an ``Integer`` and assign a default value of ``40``

### #### Add angular speed {#add-angular-speed}

To define the angular speed variable:

1. Click the <EditorIcon name="Add"/> button to add a new variable in the **Variables** comp
2. Give the variable the name ``angular_speed``, and press enter.
3. Select the variable in the **Component** view to have the **Inspector** display the varia
4. In the **Inspector**, change the ``Type`` to an ``Float`` and assign a default value of ``3.14``

:::important

Angles in Godot work in radians by default, but you have built-in functions available if you

:::

### #### Apply rotation

To move our icon, we need to update its position and rotation every frame in the game loop.

We can use the **Process** event, a Godot virtual override function, to do this.

If your orchestration extends a class that extends the ``Node`` class, like ``Sprite2D``, this i

:::important

Games work by rendering many images per second, each called a frame, and they perform this i

We measure the rate at which a game produces these images in Frames Per Second (FPS).

Most games aim for 60 FPS, although you might find values lower or higher.  
:::

We are going to add the **Process Event** node much like we did the **Init Event** node.

1. Press the `<EditorIcon name="Override"/>` button on the top-right of the **Functions** pane.
2. In the **All Actions** dialog, locate the ``Process`` event, in the **Methods > Node** category.
3. To add the node, either press **Add** or simply hit **Enter**.

With the **Process** event node on the graph, we need to connect logic to this node to perform

```
```python
rotation += angular_speed * delta
```

The above line increment's the sprite's rotation every frame. Here `rotation` is a property inherited from the `Node2D` class, which `Sprite2D` extends. This means that we can work directly with the `rotation` property in the orchestration.

To do this visually, we need to add several more nodes to our graph:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for `Get Rotation`.
3. Select the `Get Rotation` choice and either press **Add** or simply hit **Enter**.
4. Right-click the graph again to open the **All Actions** dialog.
5. Search for `Set Rotation`.
6. Select the `Set Rotation` choice and either press **Add** or simply hit **Enter**.
7. Right-click the graph area once more to open the **All Actions** dialog.
8. Search for `Multiply`.
9. Select the `Float Multiply Float` choice that is under the **Types > Float > Operators** category.
10. Either press **Add** or simply hit **Enter**.

Finally, match all the nodes as shown here. This visually represents the `rotation += angular_speed * delta` logic.

Run the scene in the editor to see the icon turn in-place.

Apply movement Let's now make the node move. In order to do this, we need to add one additional step inside the **Process** event node logic.

```
var velocity = Vector2.UP.rotated(rotation) * speed
position += velocity * delta
```

In this code, we are using a variable named `velocity` to represent the direction and speed. To make the node move forward, we start from `Vector2.UP`, a vector that simply points up, and we rotate it by calling the `Vector2` method `rotated()`. This expression, `Vector2.UP.rotated(rotation)`, is a vector that points forward relative to our icon. By multiply this by our `speed` property, it gives us a velocity that we can use to move the node. And finally we multiply

this **velocity** with the **delta** time, and add that the node's current position for it to move per frame.

To set this up visually, we need to:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Type Constant**
3. Select the **Type Constant** node and set the type as **Vector2** and the constant as **UP** in the **Inspector**.
4. Right-click the graph to open the **All Actions** dialog.
5. Search for **Rotated**, selection the choice under the **Vector2** category.
6. Next add three math operator nodes
 - Multiply **Vector2** by **Integer**.
 - Multiply **Vector2** by **Float**.
 - Addition **Vector2** by **Vector2**.
7. Finally add two function call nodes
 - Search for **Get Position**
 - Search for **Set Position**

With the nodes, we connect them as shown here:

Run the scene to see the Godot head run in circles.

Moving nodes like this does not take into account collisions with other objects. In a future tutorial, you will learn another approach to moving and detecting collisions.

Complete script

Here is what the complete `sprite_2d.torch` orchestration script looks like for reference.

Refactor and use functions

A common task in programming is to polish code so that it's easier to digest and understand. One of the common complaints of visual scripting is that it's a massive web of connections, and what we've done already is shaping up to be just that. Thankfully, Orchestrator provides multiple tools to help with this, and the first is to use user-defined functions.

Moving apply rotation to a function

The first task we're going to focus on is moving the **Apply Rotation** logic to a user-defined function called `apply_rotation`. This user-defined function will accept the **delta** time in seconds.

To create a user-defined function quickly from a set of nodes, we can use the **Collapse to Function** feature. Select the following nodes you see highlighted in a **yellow** border:

You can also hold the **Ctrl** key and left click each node separately to select multiple nodes.

Now while these nodes are selected, right-click any of the selected node's titlebar and select **Collapse to Function**. A new function will be created where you can assign its name as **apply_rotation**.

This procedure automatically replaces the nodes in the original graph with a **Call Function** node that calls the function, and places the selected nodes in the new function graph.

The last step is to left-click the **Function Entry** node, and change the input argument name from **A** to **delta**, as the collapse operation defaults to the connected target pin name.

Using the **Collapse to Function** feature automatically adds the **Function Entry** and **Function Result** nodes, and wires them accordingly. This saves time during refactoring operations.

The user-defined function, **apply_rotation**, should look like this:

Your **EventGraph** should now look similar to this:

Moving apply movement to a function

Now that we have moved the logic for rotation to a user-defined function, the next step is to do the same for **Apply Movement**. Select all nodes except **Process Event** and **Apply Rotation**, right-click one of their titlebars and select **Collapse to Function**. When prompted, assign the function's name **apply_movement**.

Open the function graph and rename **B** to **delta**.

The user-defined function graph should look like this:

The new refactored **EventGraph** should now look like this:

By refactoring the logic from the **EventGraph** into two distinct **Function** graphs, this makes the graphs more readable.

Any user-defined function can be focused by double-clicking the **Call Function** node. For example, double-clicking the **Apply Rotation** node will open the **apply_rotation** function graph, and focus the function's **Function Entry** node.

Listening for player input

Building upon the steps from Creating your first orchestration and Refactor and use functions, let's look at another important feature of any game: giving control to the player. To add this, we need to modify our **sprite_2d.torch** orchestration.

You have two main tools to process player input in Godot using Orchestrator:

- Using the built-in event callbacks, mainly **Unhandled Input Event**. Like **Process Event**, it's a built-in function call that Godot will call every time the player presses a key. It's the tool you want to use to react to events that don't happen each frame, like pressing **Space** to jump.
- Using the **Engine Singleton** node to access the **Input** singleton. A singleton is a globally accessible object. Godot provides access to several of these, and they're available in Orchestrator. It's generally the right tool for checking for input every frame.

We are going to use the **Input** singleton here as we need to know if the player wants to turn or move each frame.

Turning

For turning, we should use a new variable: **direction**. In the **Component** panel, we're also going to create a new user-defined function to calculate the direction.

Calculating the direction To add the new function, follow these steps:

1. Press the on the **Functions** panel in the **Component** view.
2. Set the new function name as `calculate_direction`.

In this function, what we ultimately want to accomplish is this logic:

```
python title="GDScript" func calculate_direction():    var direction
= 0    if Input.is_action_pressed("ui_left"):        direction =
-1    if Input.is_action_pressed("ui_right"):        direction = 1
return direction
```

Since this function will return an **Integer** value, we need to add a **Function Result** node to the graph.

1. Click on the **Function Entry** node if it isn't already selected.
2. In the **Inspector** view, click **Add Outputs**.
3. Click on the button with **Any** and search for **Integer**, setting the type to **integer**.

The `calculate_direction` graph should look like this:

Our **direction** variable is a multiplier representing the direction in which the player wants to turn. A value of 0 means the player isn't pressing the left nor the right arrow key. A value of 1 means the player wants to turn right, while a value of -1 means they want to turn left.

In Orchestrator, to check whether an action is pressed, we need to use the **Input Action** node. This node allows you to check whether a specific action mapping is *pressed*, *released*, *just pressed*, or *just released*.

1. Right-click on the graph to open the **All Actions** dialog.
2. Search for **Input Action**.
3. Select the **Input Action** choice and press the **Add** button or simply press **Enter**.

We need two of these nodes, one to check `ui_left` and another to check `ui_right`. We can duplicate an existing node by using the **Ctrl+D** shortcut.

To duplicate the **Input Action** node:

1. Select the **Input Action** node, if it isn't already selected.
2. Press the **Ctrl+D** shortcut or right-click and select the **Duplicate** option.

You should now have two **Input Action** nodes, as shown here:

With these two nodes, you need to configure each of them to check for the desired state:

1. Select one **Input Action** node.
2. In the **Inspector** panel, set the **Action** to `ui_left`.
3. Select the other **Input Action** node.
4. In the **Inspector** panel, set the **Action** to `ui_right`.

These two nodes should now look like this:

Next, to apply the two **if** conditions, we need use two **Branch** nodes.

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Branch**.
3. Select the **Branch** option and press **Add** or simply hit **Enter**.
4. Select the **Branch Node** and duplicate it, so that you have two **Branch** nodes.

The next thing we need before we can wire up these nodes is a variable to represent the `direction` value. In this case, define a variable named `direction` using the type `Integer` like had previously defined for `speed`. For `direction`, leave its default value unchanged, as it will default to 0. If you need a reminder of how to do that, see the Add angular speed section.

With the `direction` variable setup, drag the `direction` variable from the **Component** panel onto the graph 3 times. Each time, making sure to select that you want to create a **Set direction** node.

Now one of the cleanest ways to represent a sequence of operations, like checking two **if** (**Branch**) conditions followed by some logic that will run based on the prior stages is to use a **Sequence** node. In our case, we're going to create a **Sequence** node that will have 4 output execution pins, one to initialize our `direction` variable, two to handle the two **if** checks, and lastly one to handle the incrementing of the `rotation`.

1. Right-click the graph and open the **All Actions** dialog.
2. Search for **Sequence**.

3. Select the **Sequence** option and press **Add** or simply press **Enter**.
4. In the **Sequence** node, press the button so that there are **4** output execution pins. These pins will be labeled **Then 0**, **Then 1**, **Then 2**, and **Then 3**.
5. Connect the **Then 0** output execution pin to one of the **Set direction** nodes.
6. In that **Set direction** node, make sure that the **direction** value is 0 (the default).

Now, let's wire up the nodes.

1. Starting with the **Sequence** node, connect the **Then 1** output pin to one of the **Branch** nodes.
2. Connect the **Action ui_left** node's output pin to the **Condition** input pin of that **Branch** node.
3. Connect the **Branch** node's **True** output execution pin to one of the **Set direction** nodes.
4. Set the **direction** input value to -1 on the **Set direction** node.

At this point, we've taken care of setting up the first **if** condition for `Input.is_action_pressed("ui_left")`. Now, we need to repeat this process for the other **if** condition.

1. Starting with the **Sequence** node, connect the **Then 2** output pin to the **Branch** node that isn't connected.
2. Connect the **Action ui_right** node's output pin to the **Condition** input pin of that **Branch** node.
3. Connect the **Branch** node's **True** output execution pin to the **Set direction** node that isn't connected.
4. Set the **direction** input value to 1 on the **Set direction** node.

With this, the sequence part of the graph should look similar to this:

Now, you need to wire the **Function Entry** and **Function Result** nodes with this, returning the **direction**.

1. Connect the **Function Entry** output execution pin to the input execution pin of the **Sequence** node. This will break the existing connection between the **Function Entry** and **Function Return** nodes, and that's expected.
2. Drag the **direction** variable from the **Component** view, and spawn a **Get direction** node.
3. Connect the **Then 3** output pin from the **Sequence** node to the input execution pin of the **Function Result** node.
4. Finally, connect the **Get direction** node's data output to the input **Return Value** pin of the **Function Result** node.

The graph for `calculate_direction` should look like this:

Using the direction calculation The last step is to use the `calculate_direction` function inside our `apply_rotation` function.

1. Navigate to the `apply_rotation` function, double-clicking it in the **Component** view.
2. Drag the `calculate_direction` function onto the graph from the **Component** view. Be sure to select the **Add Call Function** option from the context-menu.
3. Select the **Multiplication** node that multiplies two `Float` values and press **Ctrl+D** to duplicate.
4. Connect the **Apply Rotation** entry node output execution pin to the input execution pin of the **Calculate Direction** node.
5. Connect the `delta` output pin on **Apply Rotation** to the B input of the duplicated **Multiply** node from step 3.
6. Connect the `Return value` output pin on the **Calculate Direction** to the A input of the duplicated node.
7. Connect the duplicated node's output pin to the unconnected **Multiplication** node A pin.
8. Lastly connect the **Calculate Direction** output execution pin to the **Set Rotation** input execution pin.

The final graph should look like this:

If you run the scene, the icon should rotate when pressing the **Left** and **Right** arrow keys.

If you don't press either of the arrow keys, it will simply move forward, and off the screen. So be sure to constantly press the **Left** or **Right** arrows to keep in on the screen.

Moving forward

Now, as nice as this simple game is at this point, we'd like to have better control of our Godot icon.

In this section, we're going to adjust how `apply_movement` works, and only have the icon move when the player presses the **Up** arrow key. This will make the icon easier to control and keep on on the screen.

Calculating the velocity If you recall when we originally designed the `apply_movement` function, we used the following to calculate the `velocity`.

```
python title="GDScript" var velocity = Vector2.UP.rotated(rotation)
* speed
```

Now instead, we want to conditionalize this, having the `velocity` set to `Vector2.ZERO`, and only applying the calculation if the player has pressed the **Up** arrow key.

```
python title="GDScript" var velocity = Vector2.ZERO if Input.is_action_pressed("ui_up"):
velocity = Vector2.UP.rotated(rotation) * speed
```

We're going to accomplish this by introducing a new function called `calculate_velocity`.

1. Press the button at the top of the **Functions** panel in the **Component** view.
2. When asked for a function name, enter `calculate_velocity`.

In this function, we either want to return `Vector2.ZERO` if the player has not pressed the **Up** arrow key, or return the velocity based on the current `rotation` and `speed` values. The first step we need to is to add the **Function Return** node, returning a `Vector2` value.

1. Select the **Function Entry** node.
2. In the **Inspector** view, click the **Add Inputs** button to add an argument.
3. In the **Inspector** view, set the argument name as `angle` with type `Float`.
4. In the **Inspector** view, click the **Add Outputs** button to add the return node.
5. In the **Inspector** view, set the return value's type to `Vector2` from `Any`.

Now to simplify things, navigate back to the `apply_movement` graph and select these nodes. The nodes to select are highlighted with a **yellow** border:

1. Press the **Ctrl+C** shortcut to copy the nodes into the copy buffer.
2. Navigate back to the `calculate_velocity` function.
3. Press the **Ctrl+V** shortcut to paste the nodes onto the graph.

With these nodes in the `calculate_velocity` function, you are going to use a similar technique that was used for `direction`, relying on a **Sequence** node to:

- Initialize the `velocity` value to `Vector2.ZERO`
- Check whether the `ui_up` action is pressed, and only modifying the `velocity` if `true`.
- Return the calculated `velocity` value.

Before we add nodes, you need to create a `velocity` variable. This variable should be of type `Vector2`, and its default value left unchanged. If you don't remember the steps, you can refer to the **Add angular speed** section once more.

With the `velocity` variable added, perform these steps:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Sequence**.
3. Select the **Sequence** option and press the **Add** button or simply hit **Enter**.
4. Connect the output execution pin from the **Calculate Velocity** node to the input execution pin of the **Sequence** node.
5. Drag the `velocity` variable from the **Component** panel onto the graph.
6. Connect the **Then 0** output pin on the **Sequence** node to the input execution pin on the **Set Velocity** node.

7. Open the **All Actions** dialog, and add a **Branch** node to the graph.
8. Connect the **Then 1** output execution pin to the input execution pin of the **Branch** node.
9. Open the **All Actions** dialog, and add an **Input Action** node to the graph.
10. Select the **Input Action** node.
11. In the **Inspector** view, change the **Action** property to be **ui_up**.
12. Connect the **Input Action** output pin with the input **Condition** pin on the **Branch** node.
13. Connect the **True** output pin from the **Branch** node to the input execution pin on the **Rotated** node.
14. Select the **velocity** variable in the **Component** panel and drag onto the graph, selecting **Set velocity**.
15. Connect the output execution pin from **Rotated** to the input execution pin of ****Set Velocity**.
16. Connect the output pin from the **Multiplication** ($A*B$) node to the input pin of **Set Velocity**.
17. Drag the **velocity** variable from the **Component** panel and select **Get velocity**.
18. Connect the **Then 2** output execution pin to the **Return Node** input execution pin.
19. Finally, connect the **Get velocity** output pin to the **Return Node** input **Return Value** pin.

At the end, your graph for `calculate_velocity` should look like this:

Using the velocity calculation With the `calculate_velocity` function done, we need to use this in the `apply_movement` function. Navigate back to the `apply_movement` function and perform these steps:

1. Remove the node that you copied to the `calculate_velocity` function. In case you forgot, they're the ones shown here with a **yellow** border:
2. Drag the `calculate_velocity` function from the **Component** view onto the graph. Again, select the **Add Call Function** option from the context-menu.
3. Connect the output execution pin from **Apply Movement** to the input pin on **Calculate Velocity** node.
4. Connect the output execution pin from **Calculate Velocity** to the input execution pin on **Set Position** node.
5. Finally, connect the **Vector2** output pin on **Calculate Velocity** node to the input **A** pin on the **Multiplication** node that's currently unconnected.

The new simplified `apply_movement` function should look like this:

If you now run the scene, you'll notice the following happens:

- The Godot icon does not move unless you press the **Left**, **Right**, or **Up** arrow keys.

- The **Left** arrow key rotates the icon to the left.
- The **Right** arrow key rotates the icon to the right.
- The **Up** arrow key makes the icon move forward in its current facing direction.

Summary

In summary, every script in Godot represents a class that extends one of the engine's built-in classes. The node types your classes inherit from give you access to properties, such as **rotation** and **position** in our sprite's case. You also inherit many functions, which we didn't use in this example.

In an Orchestration, using variables is an excellent way to store and manage state. Additionally, user-defined functions are a great way to organize smaller pieces of code to reduce the complexity of large visual script graphs.

And who would have thought, visual scripting can be done without being spaghetti!

All nodes

All Orchestrator visual script nodes are based on the **ScriptNode** class. The following hierarchy represents all the node types available that can be added to an **Orchestration** script.

- Script Node
 - Arrays
 - Autoloads
 - Comments
 - Constants
 - * Class-specific constants
 - * Global constants
 - * Math-specific constants
 - * Singleton-specific constants
 - * Type-specific constants
 - Dialogue
 - * Choice
 - * Show Message
 - Events
 - Flow Control
 - * Branch
 - * Chance
 - * Delay
 - * For loop
 - * For each
 - * For each key-value
 - * Random

- * Select
 - * Sequence
 - * Switch
 - * Type Cast
 - * While
- Functions
- Input
- Math
- Memory
 - * New / Create Objects
 - * Free / Destroy Objects
- Properties
 - * Get Property
 - * Set Property
- Resources
 - * Get Resource Path
 - * Preload Resource
- Scene
 - * Get Self
 - * Get Scene Node
 - * Get Scene Tree
 - * Instantiate Scene
- Signals
 - * Await Signal
 - * Emit Signal
- Singletons
- Utilities
 - * Construct/Make
 - * Destruct/Break
 - * Print String
- Variables
 - * Get Variable
 - * Set Variable

Arrays

One of Godot's built-in types is an **Array**, which is responsible for storing a collection of elements.

The Orchestrator documentation does not get into the specifics of what an **Array** is or is used for at a fundamental level. If you need more details on what an **Array** is or why you may need or want to use such a type, please see the Godot documentation.

Overview

The Godot engine exposes a variety of nodes to operate on **Array** container types, such as getting an entry by its index, removing an entry by index or value, or getting the size of the array to name a few. These operations can all be found by searching for *array* in the **All Actions** search window, some shown below:

Sample of array nodes

Orchestrator extends the Godot Engine by introducing several custom node types, denoted by the special icons:

- **Make Array**, to create a new array type
- **Add Element**, adds an element to an existing array type
- **Find Array Element**, finds an element in an existing array
- **Remove Array Element**, removes an element in an existing array
- **Remove Array Element By Index**, removes an element in an existing array by index
- **Clear Array**, clears the contents of the array
- **Append Arrays**, appends the contents of one array into the other

There are also two operator nodes for **Array** data types:

- **Get At Index**, gets an element at a given index
- **Set at Index**, sets an element at a given index

For any Godot provided **Array** action, you can view the documentation for that action in the Godot Editor by right-clicking the node in the graph and clicking the **View Documentation** option. The following sections focus on the custom nodes and the behavior they supply to Orchestrator.

Creating an array

To create a new **Array** to store elements, select the **Make Array** action in the **All Actions** search window. The node is designed to accept zero or more elements as input values. When the node is first placed onto the graph canvas, it will return an empty **Array**, as illustrated here:

Creates an empty Array

To add elements to the **Array** during construction, press the button. This will add one new input pin for each key-press that accept data types for the new element. As shown below, this is what the node will look like after pressing the **Add Pin** button just once:

Creates an Array with an Element

The input pins are wildcards, and will accept any type of connection from any other pin type.

Creating a **Typed** array is only supported in variable declarations.

Node properties

The following describes the input and output pins for the **Make Array** node.

Property	Description
Element <i>n</i>	This is the entry's <i>element</i> value.
Array	The output Array that will contain any defined Element <i>n</i> values.

Add element

If you already have an **Array** value, one of the most common operations is to add a new element to the end of the array. This can be done by using the **Add Array Item** node.

Adds a new element to an Array

Node properties

The following describes the input and output pins for the **Add Array Item** node.

Property	Description
Target	This is the target Array that will be modified by this node.
Element	The element value that will be appended to the end of the Array .
Array	The modified Array as an output value, after the element was added.
Index	The index into the Array where the new element exists.

Find element index

If you have an existing **Array** and you want to determine what index an element uses, this can be done by using the **Find Array Element** node.

Finds an element's index in an Array

Node properties

The following describes the input and output pins for the **Add Array Item** node.

Property	Description
Target	This is the target Array that will be searched.
Item	The element value to locate within the Array .
Array	The target Array as an output value for chaining operations.
Index	The index into the Array where the element exists.

If the specified **Item** does not exist in the array, the returned **Index** will be **-1**.

You can also use the built-in Godot function **has** to search the **Array** if you simply need to know if the **Array** contains the element item. The function returns **true** if the item exists and **false** if it does not.

Remove element

Another common operation is to remove an element from an **Array**. Orchestrator provides two ways to remove elements, remove by index or by value. These are described below.

Remove by index

To remove an element from an **Array** by index requires that you know in advance the index/position of the element in the container. This could be information you've stored in some data structure, or perhaps you've used the **Find element index** node.

Removes an element by index in an Array

Node properties The following describes the input and output pins for the **Remove Array Item By Index** node.

Property	Description
Target	This is the target Array that will be modified.
Index	The element index to be removed.
Array	The target Array as an output value for chaining operations, after the element was removed.

Remove by value

It's often common where you may have the element in an **Array**, but you don't know what the element's position is in the container. You could use the **Find element index** node or other functions to identify the position, but this adds unnecessary clutter to your graphs. This is where the ability to remove by value becomes valuable.

Removes an element by value in an Array

Node properties The following describes the input and output pins for the **Remove Array Item** node.

Property	Description
Target	This is the target Array that will be modified.
Element	The element to be removed.
Array	The target Array as an output value for chaining operations, after the element was removed.
Removed	An output boolean value that indicates whether the element was found and removed.

Clear array

If you need to remove all elements from an **Array**, while you could use a loop, that's highly inefficient. The most efficient way to clear an array is to use the **Clear Array** node, as shown here.

Clear/remove all elements from an Array

Node properties The following describes the input and output pins for the **Clear Array** node.

Property	Description
Target	This is the target Array that will be cleared.
Array	The target Array as an output value for chaining operations, after the array was cleared.

Append arrays

In some instances you may have multiple **Array** values where you'd like to combine their contents into a single **Array** value. This can be achieved by using the **Append Arrays** node.

Appends contents of the Source array into the Target array

Node properties The following describes the input and output pins for the **Append Arrays** node.

Property	Description
Target	This is the target Array that will be modified.

Property	Description
Source	The source Array where elements will be copied from and appended to the target Array .
Array	The target Array as an output value for chaining operations, after the array was modified.

Operators

In most programming languages, the **Array** data type has index-based operators that allow you to read or write values using the `[]` brackets combined with an index/position. Orchestrator's visual scripting language is no different.

Get element at index

The **Get Element at Index** node is the effective inverse of the **Find Array Element** node, allowing to read and obtain the value at a given index/position within the array. If the supplied index is out of bounds for the **Array**'s size, an error occurs, just like GDScript.

```
gdscript title="GDScript example of get by index" showLineNumbers
var arr : Array = ["a", "b", "c"] print(arr[1]) ## outputs "b"
```

Read the array element at a given index

Node properties The following describes the input and output pins for the **Get Element at Index** node.

Property	Description
Array	This is the Array that will be read.
Index	The index within the Array to read. <i>The value should be greater-than-or-equal to 0 and less than the Array size.</i>
Element	The element that exists at the specified Index within the Array .

Set element at index

The **Set Element at Index** node is the inverse of the **Get element at index** node, allowing a value to be written to the given index/position within the specified array. Unlike other languages where you use the `[]` operator to write the values, this node allows for resizing the array on-demand.

Write an element at a given index in the array

Node properties The following describes the input and output pins for the **Set Element at Index** node.

Property	Description
Array	This is the Array that will be modified.
Index	The index within the Array to write. <i>The value should be greater-than-or-equal to 0.</i>
Element	The element to write at the specified Index within the Array .
Size To Fit	Specifies whether the Array 's capacity will be resized automatically.
Result	The modified Array as an output value for chaining operations, after the array was modified.

Autoloads

Godot's autoload concepts

Godot's scene system, while powerful and flexible, has a drawback: there is no method for storing information (e.g. a player's score or inventory) that is needed by more than one scene.

It's possible to address this with some workarounds, but they come with their own limitations:

- You can use a “master” scene that loads and unloads other scenes as children. However, this means you can no longer run those scenes individually and expect them to work correctly.
- Information can be stored to disk in `user://` and then loaded by other scenes that require it, but frequently saving and loading data is cumbersome and inefficient.

The Singleton pattern is a useful tool for solving this common use case. In Godot's case, it's possible to reuse the same scene or class for multiple singletons, as long as they have different names.

Using this concept, you can create objects that:

- Are always loaded, no matter which scene is currently running.
- Can store global state such as player data.
- Can handle switching scenes and in-between scene transitions.
- *Act* like a singleton, since Orchestrator and GDScript neither support global variables by design.

Godot uses a concept called **Autoloads** to achieve these characteristics. You can read more about Godot's implementation in the Godot documentation.

Registering autoloads

Before you can use an **Autoload** in Godot, it must first be registered. To register an autoload:

1. Open **Project > Project Settings > Autoloads** dialog.
2. Select the path to the **Orchestration**, **GDScript**, or **C#** script.
3. Assign a unique **Node Name** for the autoload.
4. Press the **+ Add** button to register the autoload.

Accessing autoloads in orchestrations

Accessing an autoload in an **Orchestration** requires using the **Get Autoload** node. The **Get Autoload** node returns a reference to the specific Autoload as its *output* pin.

In the above example, the **Autoload Node** returns a reference to the **Game-Data** autoload.

There are two ways to add an **Autoload** node:

1. Selecting the registered autoload from the **Project > Autoloads All Actions** category list.
This choice is only permitted when right-clicking the graph; it will not be shown when dragging from an input/output pin.
2. Adding a generic **Get Autoload** node to the graph.

To add a generic **Autoload** node:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **autoload**.
3. Select the **Get Autoload** choice and press the **Add** button or simply press **Enter**.

Changing the autoload reference

If the project has only defined a single autoload, the **Get Autoload** node will default to that autoload. If you have defined multiple autoloads in your project, you may need to change the autoload reference.

To change the autoload reference:

1. Select the **Get Autoload** node.
2. In the **Inspector** view, change the **Autoload** property.

Orchestrations as autoloads

An **orchestration** can also be used as an *Autoload*, just like **.gd** and **.cs** scripts. To use an **Orchestration** as an **Autoload**, follow the Registering autoloads

procedure, and select the **Orchestration** rather than a GDScript (.gd) or C# (.cs) script files.

When adding or removing an autoload from **Project > Project Settings** autoload tab, the **All Actions** choices will automatically update to reflect the changes.

Comments

A **Comment Node** can be used for multiple purposes.

The node can be configured with both a *title* and *text* that can add informational context to parts of a graph. Additionally, comments can be used as organizational tools, to group logical parts of the graph together. This allows for moving groups of nodes together as a single unit rather than each node separately, or having to select all applicable nodes.

Creating comment nodes

Creating comment nodes requires a few short steps:

1. **Right-click** and choose **Add Comment** from the all actions window.
2. Selecting the node will display several configurable properties in the editor Inspector, including the ability to customize the titlebar text, the titlebar alignment, the node's background color along with the font size, color, and text.

Comment nodes can also be added using the keybinding **Graph Editor > Create Nodes > Create Comment**. By default, this keybinding is set to **C** but can be customized in the Orchestrator's **File > Settings > Shortcuts** tab.

Editing comment node properties

There are two ways to manage the properties for a **Comment Node**. The first approach is to use the **Inspector** view in the editor, all public configurable properties are available there.

The second is to right-click the comment node's titlebar, showing a context menu of options:

See the Properties section for details on what node properties can be customized.

Deleting comment nodes

Clicking on any comment node and pressing the **Delete** key, or **Right-Click** and selecting **Delete** from the pop-up menu will remove the comment node from the graph.

Removing a comment node will have no impact on any nodes that it contains, as long as the nodes are not selected. The removal of the comment node unbinds any attached nodes to the frame automatically, making the nodes ungrouped.

Attach/Detach nodes

The **Comment Node** uses Godot's native **GraphNode** behavior. This means that when you drag a node over the top of a comment node, the frame will automatically join that node to the frame, grouping it with any other nodes the frame contains.

To ungroup a node from a comment node, right-click the node's titlebar and access its context-menu. There will be an option called **Detach from Frame**, selecting it will detach the node and allow you to move it outside the frame's boundaries.

Auto-shrink

By default, **Comment Nodes** enable the auto-shrink behavior, meaning that moving any nodes that are contained within the comment grow or shrink the comment node's boundaries accordingly. When auto-shrink is disabled, the comment node's resizer tool becomes available in the bottom right-side of the comment node's frame.

The auto-shrink behavior can be disabled by toggling the comment node's auto-shrink behavior off either in the **Inspector** view or by the comment node's context-menu options.

Properties

Comment nodes have a number of properties that can be adjusted in the **Inspector** view. Simply select the comment node to show its properties in the **Inspector** view.

Property	Description
Icon	Specifies a custom icon to be placed in the top-right section of the comment node's title bar.
Titlebar Text	The text that will be shown in the comment node's title bar.
Align Center	If this is checked, the titlebar text will be center aligned.
Background Color Enabled	This allows to control whether the background color is applied to the comment node.

Property	Description
Background Color	This allows you to change the background color of the comment node. Remember to use Alpha to adjust the transparency of the background.
Font Size	This allows specifying the text size of the text in the node's panel area.
Text Color	This allows you to change the text color of the text in the node's panel area.
Comments	This allows you to provide an extended description using a multi-lined text area that will be added to the node's panel area.
Autoshrink Enabled	This controls whether the comment node auto-shrinks based on the nodes it contains, or whether it can be freely resized manually.

Constants

Constants are an integral part of any programming language, as they provide a human-readable names to special values. For example, in mathematics, there are numerous constants called **TAU** and **PI**, and Orchestrator supports these and many more.

In this section, you will learn what are Orchestrator's constant types, and how to use them. There are different constant types, depending on your needs, described below.

Constant nodes are considered *experimental*, and may change in the future. While it is safe to use these in your Orchestrations, they may be replaced by a more consolidated, and user-friendly node in the future.

Global constants

Global constants are one of the most popular types of constants. They are in the global namespace and do not require the use of a prefix to access. For example, if your game needs base behavior around the constant for the **ESC** key, you would use the **KEY_ESCAPE** global constant.

To access global constants in Orchestrator:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Global constant**
3. Press the **Add** button or simply press **Enter**.

Once the node has spawned, select the node in the **Graph** and adjust the properties in the **Inspector** view.

Property	Description
Constant	The constant value to output from the node when executed, i.e. KEY_ESCAPE.

The output pin for a **Global Constant** is an enumeration, but Godot represents this as a logical integer.

Math-specific constants

Godot specifically registers math-specific constants slightly differently, therefore in order to access constants such as **One**, **PI**, **PI/2**, **TAU**, and others, you must use a **Math Constant** node.

To access math constants in Orchestrator:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Math constant**
3. Press the **Add** button or simply press **Enter**.

Once the node has spawned, select the node in the **Graph** and adjust the properties in the **Inspector** view.

Property	Description
Constant	The constant value to output from the node when executed, i.e. PI.

The output pin for a **Math Constant** node is always a float.

Class-specific constants

Class-specific constants are values that must be accessed by using the class as a prefix. For example, if your game needs to access the **SYSTEM_DIR_DESKTOP**, you would do so by using the **OS** class.

```
gdscript showLineNumbers title="GDScript example of class-constant access" var desktop_directory = OS.SYSTEM_DIR_DESKTOP
```

To access a class-specific constant in Orchestrator:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Class constant**.
3. Press the **Add** button or simply press **Enter**.

Once the node has spawned, select the node in the **Graph** and adjust its properties in the **Inspector** view.

Property	Description
Class Name	The Godot class that owns the constant, i.e. <code>OS</code> .
Constant	The constant value to output from the node when executed, i.e. <code>SYSTEM_DIR_DESKTOP</code> .

The output pin for a **Class Constant** node is always a integer.

Singleton-specific constants

Singleton-specific constants are very similar to class-specific constants; however, these are related specifically to Godot's registered singleton objects, such as `Input` or `AudioServer`. For example, to access `MOUSE_MODE_HIDDEN` or `MOUSE_MODE_CAPTURED`, you would use a **Singleton Constant** node.

To access singleton constants in Orchestrator:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Singleton constant**
3. Press the **Add** button or simply press **Enter**.

Once the node has spawned, select the node in the **Graph** and adjust the properties in the **Inspector** view.

Property	Description
Class Name	The singleton class to scope the constants based upon, i.e. <code>Input</code> .
Constant	The constant value to output from the node when executed, i.e. <code>MOUSE_MODE_HIDDEN</code> .

The output pin for a **Singleton Constant** node is always a integer.

Type-specific constants

Type-specific constants are similar to class-specific constants; however, these are related specifically to Godot's built-in data types, like `Vector3`. For example, if you want to use `Vector3.ZERO` to set a `Vector3` to its default value, you would use a **Type Constant** node.

To access type constants in Orchestrator:

1. Right-click the graph to open the **All Actions** dialog.
2. Search for **Type constant**
3. Press the **Add** button or simply press **Enter**.

Once the node has spawned, select the node in the **Graph** and adjust the properties in the **Inspector** view.

Property	Description
Basic Type	The Godot built-in type to scope the constants based upon, i.e. Vector3 .
Constant	The constant value to output from the node when executed, i.e. ZERO .

The output pin for **Type Constants** varies, but it most often mirrors the Godot built-in data type.

Dialogue

Any RPG game relies heavily on a solid foundation for storytelling, providing an immersive experience for the player. One of the many tools that helps facilitate that is a conversation system to drive the interaction between the player and the story.

Orchestrator provides two key nodes to support conversation:

- Show Message node
- Dialogue Choice node

Show message node

The **Show Message** node is the foundation of the dialogue and RPG conversation system. It allows you to define who is speaking, and the text they're going to say. Additionally, you can define any number of choice inputs that allow the player to control the execution flow throughout the conversation.

In a future update, character definitions will be added to easily customize portrait, sounds, and other dialogue-features.

Using a custom scene

Orchestrator provides a default basic scene for conversations in the `res://addons/orchestrator/scenes` directory. While you may edit this scene directly, we **strongly recommend** that you make a copy and customize the scene in a different path.

To use a custom scene with the conversation system:

1. Locate your **Show Message** node(s) in the graph.
2. Click on the **Default Scene** button for the **Scene** property.
3. Locate your new custom scene file in the project, and press **Open**.

Rather than setting the custom scene per node, this can be set once in the plugin's settings.

From the Orchestrator tab, navigate to **File > Settings** and locate **Runtime > Dialogue > Default Message Scene**. Locate the custom scene file you would like to be the default, and set the file path.

Once set, all **Show Message** nodes that refer to *Default Scene* will use the scene specified by this setting.

Non-linear conversation flow

One of the most powerful ways to create immersive RPG conversation is to provide options when interacting with non-player characters, creating responses that are tailored based on the choices selected by the player. Orchestrator's **Show Message** node provides for this using the button.

To add one or more conversation choices:

1. Press the button to add a conversation choice.
2. Add a **Dialogue Choice** node to the graph and wire its output pin to the input pin of the message node's choice.
3. Connect any output logic to that choice's output pin, which will receive the output pulse when the player selects that choice.

Use choice output pins in **Show Message** nodes to alter the global game state can that influence future interactions with non-player characters for a truly immersive experience.

Properties

The following properties can be set on the **Show Message** node:

Pins	Description
Speaker	The name of the character that is currently speaking the Message text.
Message	The text being spoken by the Speaker .
Scene	The file path to the custom scene to be rendered when the node executes.
Choice n	Optional choices that can be selected by the player when interacting with the non-player character.

If no choices are specified, the player will be prompted to press a **Continue** button when using the default scene.

Dialogue choice node

The **Dialogue Choice** node is a special type of node that describes a specific choice that can be picked by the player when interacting with a **Show Message** node. Therefore, for **Dialogue Choice** nodes to be useful, they must be paired with a **Show Message** node.

In the above example, there are two **Dialogue Choice** nodes that are connected to a **Show Message** node. When the **Show Message** node executes, each choice's **visible** input condition is evaluated to determine whether that specific choice should be shown to the player. This can be extremely useful for creating varied responses and actions the player can take based on game state.

Pins	Description
Text	The text to be displayed in the conversation dialogue window for that specific choice.
Visible	Whether the this choice should be visible to the player
Choice	The output pin that should be connected to one of the Choice n input pins of a Show Message node.

Dictionaries

One of Godot's built-in types is a **Dictionary**, which is responsible for storing key-value pairs. Dictionaries are associative containers that preserve insertion order. In other languages, these are often referred to a hash map or an associative array.

The Orchestrator documentation does not get into the specifics of what a **Dictionary** is or is used for at a fundamental level. If you need more details on what a **Dictionary** is or why you may need or want to use such a type, please see the Godot documentation.

Overview

The Godot engine exposes a variety of nodes to operate on **Dictionary** container types, such as getting an entry by key, removing an entry by key, or getting the size of the dictionary to name a few. These operations can all be found by searching for *dictionary* in the **All Actions** search window, some shown below:

Sample of dictionary nodes

Orchestrator extends the Godot Engine by introducing two custom node types, denoted by the special icons:

- **Make Dictionary**, to create a new dictionary type
- **Set**, to set a dictionary key/value pair

For any Godot provided **Dictionary** action, you can view the documentation for that action in the Godot Editor by right-clicking the node in the graph and clicking the **View Documentation** option. The following sections focus on the custom nodes and the behavior they supply to Orchestrator.

Creating a dictionary

To create a new **Dictionary** to store key/value pairs, select the **Make Dictionary** action in the **All Actions** search window. The node is designed to accept zero or more key/value pairs. When the node is first placed onto the graph canvas, it will return an empty **Dictionary**, as illustrated here:

Creates an empty Dictionary

To add key/value pairs to the **Dictionary** during construction, press the button. This will add two new input pins for each key-press that accept data types for the new key and its associated value. As shown below, this is what the node will look like after pressing the **Add Pin** button just once:

Creates a Dictionary with a Key and Value

The input pins are wildcards, and will accept any type of connection from any other pin type.

Creating a **Typed** dictionary is only supported in variable declarations.

Node properties

The following describes the input and output pins for the **Make Dictionary** node.

Property	Description
Key <i>n</i>	This is the entry's <i>key</i> value that will be used to uniquely identify the value in the Dictionary .
Value <i>n</i>	This is the entry's <i>value</i> , that will be associated with its match Key <i>n</i> in the Dictionary .
Dictionary	The output Dictionary that will contain any defined Value <i>n</i> values associated by the Key <i>n</i>

When using the **Make Dictionary** node, each **Key *n*** should be unique. If multiple input key/value pairs use the same **Key**, only the last pair with the highest index *n* for **Key *n*** will remain.

Setting value in an existing dictionary

In languages like **GDScript** or **C#**, the designer uses subscript operators to assign values into a **Dictionary**. These operators use the key to effectively insert

or displace any existing entry with that key in an already existing dictionary. Here's a trivial example:

```
gdscript showLineNumbers var the_dict : Dictionary the_dict[the_key]
= the_value
```

In visual scripting, there isn't a concept of subscript operators, so this operation must be performed using a custom node type that handles the intrinsic subscript manipulation for you. This is precisely what the **Set Dictionary Item** node does.

Set a dictionary key/value item node

This node is extremely powerful as it provides a variety of functionality and information about the operation as outputs. See the node properties section for details on the input and output pins.

Node properties

The following describes the input and output pins for the **Set Dictionary Item** node.

Property	Description
Target	This is the target Dictionary that will be modified by this node.
Key	This is the dictionary key (a wildcard) to be inserted or modified by this node.
Value	This is the value (a wildcard) associated with the key that will be added by this node.
Dictionary	This is the modified Dictionary , after the key has been added by the node.
Replaced	This represents whether an existing entry with the Key was replaced if the value is true .
Old Value	This is the old value (a wildcard) that was replaced for the Key , if the Replaced output pin is true .

Events

Events are nodes that are called directly by the Godot game engine, to begin the execution of logic within a **Graph**. They enable orchestrations to perform a series of actions in response to well-defined callbacks within the Godot game engine, such as when the node is *ready*, *enters the tree*, *receives input*, and more.

Events are used to implement new functionality or to override or alter the default functionality of Godot objects. Any number of **Events** can be used within a single **Graph**, although you can only define one event node of each event type.

An event can only execute a single, linear series of execution steps. If you want to trigger multiple actions from one event, you will need to string them linearly together.

To add an event to the **Graph**:

1. Press the button in the **Functions** section of the **Components** view.
2. Search for the desired Godot event to override in the search box, i.e. **ready**.
3. Press the **Add** button or simply press **Enter**.

Once the function override has been added, connect your behavior or add behavior to the output pins of the event node.

Init event

The **Init Event** is called when the object's script is instantiated, often times after the object has been initialized in memory. This method is very similar to a *constructor* in most programming languages.

This is useful when you need to perform some initialization steps when the Orchestration is first created.

Orchestrator does not currently support arguments in the `init()` event callback.

Notification event

The **Notification Event** is called when the owning node, and by extension the Orchestration, receives a specific notification from Godot. Notifications are identified by a **what** numeric constant, such as `NOTIFICATION_PREDELETE`.

Normally an Orchestration will prefer to use one of the other overridable events rather than using the **Notification Event**.

Enter tree event

The **Enter Tree Event** is called when the Orchestration's owning node is added as a child to a node in the scene tree. You will use this event to define behavior that should happen everytime the owning node enters the scene.

Exit tree event

The **Exit Tree Event** is called when the Orchestration's owning node is removed from the scene tree. You will use this event to define behavior that should happen everytime the owning node leaves the scene, such as clean-up specific resources that may have been allocated during the script's lifetime.

Input event

The **Input Event** is called when the Orchestration's owning node receives some type of *input*. The output pin **event** will provide the orchestration with the input event that was received by the engine.

Physics process event

The **Physics Process Event** is called during the physics processing in during the main game loop. This event is synchronized with the physics frame rate, i.e. the **delta** value should always be constant and in **seconds**.

This event callback is great to perform logic that should be synchronized with physics.

If the owning node does not have physics enabled, the **Physics Process Event** will not be called. Additionally, if the owning node has been removed from the scene, this event will not be called.

Process event

The **Process Event** is called during the processing step of the main game loop. This event is fired every frame and as fast as possible, meaning that **delta** will not be consistent and can vary. The **delta** represents the time since the previous frame, and is represented in **seconds**.

This event callback is great to perform per-frame (per tick) logic.

If the owning node does not have processing enabled, the **Process Event** will not be called. Additionally, if the owning node has been removed from the scene, this event will not be called.

Ready event

The **Ready Event** is called when the owning node, and by extension the Orchestration, is "ready". Godot implies readiness when the owning node, and any children, have all entered the scene and their respective **Enter Tree Event** callbacks have executed.

This event callback is typically used for initialization, especially if the behavior expects the Orchestration to exist in the scene.

Godot executes the **Ready Event** of child nodes before parent nodes.

This callback is only fired once for each Orchestration. If the Orchestration's owning node is removed from the scene and later added, the **Ready Event** is not called a subsequent time. If the Orchestration's **Ready Event** should be called again when added back to the scene, a prior call to the Orchestration's `request_ready()` function must be made.

Shortcut input event

The **Shortcut Input Event** is called as part of the Input event flow, giving you a chance to handle the input as a **Shortcut**. Please see the Godot documentation on how Input works and at what stages events fire.

Unhandled input event

The **Unhandled Input Event** is called when an **InputEvent** has not been consumed by any **_input** or **UI Control** objects. It is called after the **Unhandled Key Input Event**, and is called in a bottom up order through the scene tree. The output pin **event** contains the received **InputEvent** object.

This callback is an excellent place to handle non-key related input, such as mouse interactions.

If the owning node has disabled processing of unhandled input via **set_process_unhandled_input** with **false**, this method is not called. Additionally, if the owning node has been removed from the scene, this event will not be called.

Unhandled key input event

The **Unhandled Key Event** is called when an **InputKeyEvent** has not been consumed by any **_input** or **UI Control** objects. It is called before the **Unhandled Input Event**, and is called in a bottom up order through the scene tree. The output pin **event** contains the received **InputKeyEvent** object.

This callback is an excellent place to handle key-related input, such as key pressed or releases.

If the owning node has disabled processing of unhandled input via **set_process_unhandled_key_input** with **false**, this method is not called. Additionally, if the owning node has been removed from the scene, this event will not be called.

Custom Events

Other game engines provide the ability add user-defined custom events. In Godot, these are known as **Signals**. Please see the Signals section for details on how to add and work with signals using Orchestrator.

Flow Control

Flow Control nodes are some of the most common types of nodes used in visual scripting, as they provide the control over the sequence of how an Orchestration's nodes are executed. In other programming languages, these include constructs like if/else or branches, loops, and others.

In this section we'll talk about each **Flow Control** node Orchestrator provides.

Branch - If/Else

The **Branch** node serves as a simple way to create decision-based flow from a single true/false condition. Once executed, the Branch node looks at the incoming value of the attached boolean input, and outputs an execution pulse down the appropriate output.

In this simple example, the branch is checking the current state of a boolean variable. If **true**, it sets the color of a light to be red. If **false**, it sets the color to blue.

Node properties

The following describes the input and output pins for the **Branch** node.

Pin	Description
Condition	Takes a boolean value used to indicate which output pin that will be triggered.
True	This outputs an execution pulse if the incoming condition was true .
False	This outputs an execution pulse if the incoming condition was false .

Chance

The **Chance** node works by calculating a value between 0 and 100, and sending the execution out one of two output pins. The percentage chance is set in the **Inspector** view by setting the value of **Chance** to a value between 0 and 100.

- If the random value generated is between 0 and the specified **Chance**, the top output pin will be chosen.
- If the random value generated is greater than the **Chance** value and less-than equal to 100, the bottom output pin will be chosen.

This provides a simple, yet effective, random binary choice decision path.

Node properties

The following describes the input and output pins for the **Chance** node.

Pin	Description
Top Output Pin	The output pin that emits a pulse if the random value is between 0 and Chance (chance inclusive).

Pin	Description
Bottom Output Pin	The output pin that emits a pulse if the random value is between Chance and 100 (chance exclusive).

Delay

A **Delay** node works by temporarily yielding the execution of the orchestration without blocking the game loop. This is done by using coroutines and Godot signals to *await* on the internal timer's `timeout` callback.

```
gdscript title="GDScript example of Delay using await" showLineNumbers
func my_function() -> void: print("Hello") await get_tree().create_timer(1).timeout
## Outputs this after 1 second have passed. ## This happens
without blocking the game's render loop due to coroutine behavior.
print("Function completed")
```

Using delay in functions

Unlike an event graph, when using **Delay** within a user-defined function, special consideration must be taken.

Godot natively supports the use of the `await` keyword within top-level functions like `_ready`. However, as soon as you use the **Delay** node, which maps to `await` within a user-defined function, any location that calls that function must also be **Awaited**. For example, in GDScript:

```
“gdscript title=“GDScript example of Delay using await” showLineNumbers
func _ready() -> void: await my_function()

func my_function() -> void: print(“Hello”) await get_tree().create_timer(1).timeout
## Outputs this after 1 second have passed. ## This happens without
blocking the game’s render loop due to coroutine behavior. print(“Function
completed”)
```

Orchestrator handles this use case by specifying that a user-defined function is an ****Awaited****. This can be done by right-clicking the call function node, as shown below, and enabling the

<Figure image="/img/nodes/flow-control/delay-node-example.png" caption="How to await a user-

When a user-defined function is marked for ****Awaited****, it will be indicated by an <EditorIcon

```
### Node properties {#node-properties-delay}
```

The following describes the input and output pins for the ****Delay**** node.

Pin	Description
-----	-------------

:-----	:-----
<EditorIcon name="float"/> Duration	Specifies the number of seconds to yield the orchest

For loop

<Figure image="/img/nodes/flow-control/for-loop-node.png" caption="For Loop node"></Figure>

The **For Loop** node works like a ``randi`` loop in GDScript. It emits an execution pulse for each index between the given first and last index value out. Once the loop finishes, an execution pulse is output the **Completed** pin.

In the following simple example, the loop is triggered when the player touches a simple level trigger. The loop iterates 10 times, each time calling a **Print String**, logging the message "Itera

<Figure image="/img/nodes/flow-control/for-loop-node-example.png" caption="For Loop node exa

:::note

The standard **For Loop** node does not provide a way to break the loop early.

If you need loop ``break`` behavior, see [``For loop with break``](#for-loop-with-break).

:::

Node properties {#node-properties-for-loop}

The following describes the input and output pins for the **For Loop** node.

Property	Description
:-----	:-----
<EditorIcon name="int"/> First Index	Takes an integer representing the first index in the loop.
<EditorIcon name="int"/> Last Index	Takes an integer representing the last index in the loop.
Loop Body	This outputs an execution pulse on each iteration of the loop.
<EditorIcon name="int"/> Index	This outputs the current index in the loop.
Completed	This outputs an execution pulse when the loop has finished.

For loop with break

<Figure image="/img/nodes/flow-control/for-loop-break-node.png" caption="For Loop with Break

The **For Loop With Break** node is an enhanced version of the **For Loop** node, allowing t

In this simple example, the loop is triggered when the player touches a simple level trigger. The loop iterates 1000 times, each time hitting a Branch which checks if the loop has hit 500. If it has not, then a message with the current iteration is placed on the screen. Once it exceeds 500, the Branch sends a pulse into the **Break** pin, which breaks the loop.

<Figure image="/img/nodes/flow-control/for-loop-break-node-example.png" caption="For Loop w

Node properties {#node-properties-for-loop-with-break}

The following describes the input and output pins for the **For Loop With Break** node.

Property	Description
<EditorIcon name="int"/> First Index	Takes an integer representing the first index in the collection.
<EditorIcon name="int"/> Last Index	Takes an integer representing the last index in the collection.
Break	This execution input pin breaks the loop when triggered.
Loop Body	This outputs an execution pulse on each iteration of the loop.
<EditorIcon name="int"/> Index	This outputs the current index in the loop.
Completed	This outputs an execution pulse when the loop has finished.
Aborted	This outputs whether the loop broke early due to the break input.

For each loop

The **For Each Loop** node works similarly to the **For Loop** node, except that it iterates over a collection.

Orchestrator provides the node in two collection flavors, and each flavor has a variant that can break the loop.

Node	Iterates	Outputs
For Each	`Array`	Element, Index
For Each With Break	`Array`	Element, Index
For Each Key-Value	`Dictionary`	Key, Value
For Each Key-Value With Break	`Dictionary`	Key, Value

:::note

The collection a node iterates over is fixed when the node is created.

An existing **For Each** cannot be switched between an `Array` and a `Dictionary`; add a new node instead.

:::

Array {#for-each-loop-array}

<Figure image="/img/nodes/flow-control/for-each-loop-node.png" caption="For Each Loop node" />

The **For Each** node iterates over an `Array`, emitting an execution pulse out the **Loop Body** pin. Once every element has been visited, an execution pulse is output the **Completed** pin.

<Figure image="/img/nodes/flow-control/for-each-loop-node-example.png" caption="For Each Loop node example" />

In the above example, the loop is triggered when a player touches a simple level trigger. The loop iterates over an `Array`, output each element to the screen.

Once all elements in the array have been iterated, the loop exits through the **Completed** pin.

:::note

The ****For Each**** node does not provide a way to break the loop early.

If you need loop `break` behavior, see [`Array with break`](#for-each-loop-array-with-break)
:::

Node properties {#node-properties-for-each-loop}

The following describes the input and output pins for the ****For Each**** node.

Property	Description
<EditorIcon name="Array"/> Array	The `Array` to be iterated.
Loop Body	This outputs an execution pulse for each element in the array.
<EditorIcon name="Variant"/> Element	This outputs the current array element.
<EditorIcon name="int"/> Index	This outputs the current index in the loop.
Completed	This outputs an execution pulse when the loop has completed.

Array with break {#for-each-loop-array-with-break}

<Figure image="/img/nodes/flow-control/for-each-loop-break-node.png" caption="For Each Loop Break Node" />

The ****For Each With Break**** node works in a very similar manner to the ****For Each**** node, except it includes a **Break** input pin.

In this simple example, the loop is triggered when the player touches a simple level trigger. The loop iterates for each element in the array, each time hitting a Branch which checks if the element is in the array. If it has not, then a message with the current element is placed on the screen. Once the second element is reached, the Branch sends a pulse into the ****Break**** pin, which breaks the loop.

<Figure image="/img/nodes/flow-control/for-each-loop-break-node-example.png" caption="For Each Loop Break Node Example" />

Node properties {#node-properties-for-each-loop-with-break}

The following describes the input and output pins for the ****For Each With Break**** node.

Property	Description
<EditorIcon name="Array"/> Array	The `Array` to be iterated.
Break	This execution input pin breaks the loop when triggered.
Loop Body	This outputs an execution pulse for each element in the array.
<EditorIcon name="Variant"/> Element	This outputs the current array element.
<EditorIcon name="int"/> Index	This outputs the current index in the loop.
Completed	This outputs an execution pulse when the loop has completed.
Aborted	This outputs whether the loop broke early due to the Break pin.

Key-Value {#for-each-loop-key-value}

<Figure image="/img/nodes/flow-control/for-each-loop-key-value-node.png" caption="For Each Loop Key-Value Node" />

The **For Each Key-Value** node iterates over a `Dictionary` rather than an `Array`, emitting an execution pulse for each entry. Once every entry has been visited, an execution pulse is output the **Completed** pin.

<Figure image="/img/nodes/flow-control/for-each-loop-key-value-node-example.png" caption="For Each Key-Value Node Example" />

In the above example, the loop iterates over a `Dictionary` of item names and quantities, printing the quantity for each item. Once all entries in the dictionary have been iterated, the loop exits through the **Completed** pin.

Note
A `Dictionary` preserves the order in which entries were added, so the loop visits each entry in order.

Unlike the `Array` flavor, there is no **Index** output; use the **Key** to identify the current entry.

Note
The **For Each Key-Value** node does not provide a way to break the loop early.

If you need loop `break` behavior, see [`Key-Value with break`](<#for-each-loop-key-value-with-break>).

Node properties {#node-properties-for-each-loop-key-value}

The following describes the input and output pins for the **For Each Key-Value** node.

Property	Description
<EditorIcon name="Dictionary"/> Dictionary	The <code>Dictionary</code> to be iterated.
Loop Body	This outputs an execution pulse for each entry.
<EditorIcon name="Variant"/> Key	This outputs the current dictionary key.
<EditorIcon name="Variant"/> Value	This outputs the value that the current Key points to.
Completed	This outputs an execution pulse when the loop has finished.

Key-Value with break {#for-each-loop-key-value-with-break}

<Figure image="/img/nodes/flow-control/for-each-loop-key-value-break-node.png" caption="For Each Key-Value With Break Node Example" />

The **For Each Key-Value With Break** node works in a very similar manner to the **For Each Key-Value** node.

In this simple example, the loop iterates over each entry in a `Dictionary`, each time hitting the **Loop Body** pin. If the value is within the expected range, then a message with the current **Key** is placed into the **Output** pin. Once a value falls outside that range, the Branch sends a pulse into the **Break** pin, which exits the loop.

<Figure image="/img/nodes/flow-control/for-each-loop-key-value-break-node-example.png" caption="For Each Key-Value With Break Node Example" />

Node properties {#node-properties-for-each-loop-key-value-with-break}

The following describes the input and output pins for the **For Each Key-Value With Break** node.

Property	Description
<EditorIcon name="Dictionary"/> Dictionary	The `Dictionary` to be iterated.
Break	This execution input pin breaks the loop when reached.
Loop Body	This outputs an execution pulse for each entry in the dictionary.
<EditorIcon name="Variant"/> Key	This outputs the current dictionary key.
<EditorIcon name="Variant"/> Value	This outputs the value that the current key points to.
Completed	This outputs an execution pulse when the loop has finished.
Aborted	This outputs whether the loop broke early due to a break.

Random

<Figure image="/img/nodes/flow-control/random-node.png" caption="Random node"></Figure>

A **Random** node allows specifying one or more output execution pins, and one will be chosen at random. Each output control flow pin has an equal chance to be picked.

To add a new output execution, simply click the <EditorIcon name="ZoomMore"/> button to add a new pin. To remove an output execution, simply right-click the name of the choice and select **Remove**.

Node properties {#node-properties-random}

The following describes the input and output pins for the **Random** node.

Pin	Description
Choice_n	A random choice output execution between 0..n depending on the number of random choices.

Select - Ternary {#select}

<Figure image="/img/nodes/flow-control/select-node.png" caption="Select node"></Figure>

A **Select** node acts like a *gate* or *ternary* operator, emitting one of two different inputs. If the condition is `true`, the **Select** node allows the value of `A` to be sent as the output. If the condition is `false`, the **Select** node allows the value of `B` to be sent as the output.

```
``gdscript title="GDScript example of ternary selection" showLineNumbers
var result = a if condition else b
```

Changing pin types

By default, the **Select** node uses the Godot wildcard type to allow any value to be attached. The input pin types can be restricted by right-clicking on the A or B input pins, and selecting **Change Pin Type** in the context-menu, shown below:

By selecting a type, such as **Integer**, the pin types will change from to .

Node properties

The following describes the input and output pins for the **Select** node.

Pin	Description
A	One of two values that will be output if the Pick A condition is true .
B	One of two values that will be output if the Pick A condition is false .
Pick A	Controls whether the value of A or B is emitted as the Return Value
Result	The output value of the gate, depends on what the Pick A condition is.

Sequence

A **Sequence** node allows the execution of two or more execution chains in sequential order. This is done by first executing the connections of **Then 0**, followed by **Then 1**, and so on until all output connections have been called.

Additionally, new output connections can be added by pressing the button on the graph node. If an output connection should be removed, right-click the output pin name and select **Remove Pin**.

Unlike most control flow nodes that create nested scopes, the **Sequence** node does not. The **Sequence** node is purely for organizational purposes, and consolidates all output pin control flows into the same scope.

Node properties

The following describes the input and output pins for the **Sequence** node.

Pin	Description
Then 0	The first output connection chain to be executed, and cannot be removed.
Then 1	The second output connection chain to be executed, and cannot be removed.
Then n	The next output connection chain to be executed, any chain where $n \geq 2$ can be removed.

Switch nodes

A switch node reads a data *input*, and based on that value, sends the execution flow out to the matching (or optional default) execution *output* pin.

There are several types of switches available: **General**, **Integer**, **String**, and **Enumeration**.

General Switch

The general **Switch** node is the equivalent to a GDScript's **match** keyword. For every value you would want to match, press the button to add new **Case n** input pins, as shown here:

If the **Value** pin matches one of the values connected to any of the **Case n** pins, the control flow temporarily goes out the associated case's output pin to the right. This allows for you to specify specific logic to execute when that case matches. If none of the **Case n** pins match, the control flow temporarily goes out the **Default** pin on the top right.

When all control flow has executed on the specific **Case n** or **Default** output pin, control flow returns to the **Switch** node, and exits the node through the **Done** pin.

Node properties The following describes the input and output pins for the **Switch** node.

Pin	Description
Value	The value to be matched against zero or one of the supplied cases.
Case n	One or more values to be matched against the supplied Value input.
Done	When control returns to the Switch , this pin will be pulsed. All logic after the Switch should be added here.
Default	If none of the Case n values matched, this pin will be pulsed to execute the default logic. After the logic ends, control is returned to the Switch .
Case n Output	If the Case n value matches, this pin will be pulsed to execute that branch's logic. After the logic ends, control is returned to the Switch .

Enumeration Switch

The **Switch On Enumeration** node is a general node that allows changing the execution's control flow based on a specific enumeration type's value. In Godot, there are hundreds of enumeration types like **ClockDirection**, **DisplayServer.VSyncMode**, and more.

Unlike other switch nodes, the **Switch on Enumeration** node has no **Default** or **Done** output pins. This means that if you have logic that all branches should converge toward, it's up to you to connect those later in the branch paths.

The output execution pins for **Switch on Enumeration** nodes are auto-generated based on the enumeration's properties.

Integer Switch

The **Switch on Integer** node provides an advanced branch-scenario based on a gapless sequence of numeric values. The sequence start position is dictated by the **Start Index**, which can be set in the **Inspector** view.

Specifying the **Start Index** as 1 and then pressing the button will add an output pin labeled 1.

Pressing the once more will add another output pin labeled 2, and so on.

When right-clicking and selecting **Remove pin** for any of the output numeric case pins, any pins that come afterward will have their value decreased by 1. This is expected because the **Switch on Integer** node is designed to be a gapless, sequence of numeric values.

In addition, the **Switch on Integer** node **Default** output pin (aka else) can be toggled on or off.

- When the *input* value matches the output pin's integer label, that output pin will be pulsed.
- If none of the output pin label's match the input value, and **Default** is enabled, the **Default** pin will be pulsed.
- If none of the output pin label's match the input value, and the **Default** pin is not enabled, execution ends.

Node properties The following describes the input and output pins for the **Switch on Integer** node.

Pin	Description
Value	The value to be matched against zero or one of the supplied cases.
n	Optional output value match cases. If label matches, the pin is pulsed.
Default	If enabled, this pin is pulsed if none of the optional output value cases match.

String switch

The **Switch on String** node provides an advanced branch-scenario based on string values matched to user-defined cases.

In the **Inspector** view, the **Switch on String** node can specify whether string matches are or are not case-sensitive. In addition, the **Default** pin can be enabled or disabled based on needs.

To introduce new case match options, pressing the button will add a new **Case_n** output pin, as seen below.

The pin label can be customized in the **Inspector** view, by expanding the **Pin Names** section, and adjusting their values:

One the pin names are changed, the **Switch on String** node will reflect the changes immediately:

Repeat the process for any additional output pins you would like to add. To remove an output execution pin, right-click on the pin name and select **Remove Pin**.

Node properties The following describes the input and output pins for the **Switch on String** node.

Pin	Description
Value	The value to be matched against zero or one of the supplied cases.
Case_n / Custom Label	Optional output value match cases. If label matches, the pin is pulsed.
Default	If enabled, this pin is pulsed if none of the optional output value cases match.

Type cast - Is

A **Type Cast** node is a special type of flow control type that allows for checking the input value's logical type. When the input value's type matches the configured type in the node, the **Yes** output pin will receive the output pulse, and the **as ...** output data pin will contain the value cast to the desired type. When the input value's type does not match the configured type in the node, the **No** output pin will receive the output pulse, and the **as ...** output data pin will be **null**.

In GDScript, this is typically represented by one of two approaches:

```
gdscript title="GDScript using cast" showLineNumbers func my_function(node : Node3D) -> void:
    var camera : Camera3D = node
    if camera:
        ## The input node is a Camera3D
    else:
        ## The input node is not a Camera3D
```

```
gdscript title="GDScript using is" showLineNumbers func my_function(node : Node3D) -> void:
    if node is Camera3D:
        ## The input node is a Camera3D
    else:
        ## The input node is not a Camera3D
```

Changing the selected cast type

To change the type in the cast operation:

1. Select the **Type Cast** node in the graph.
2. In the **Inspector** view, press the button in the **Type** property.
3. Select the desired type from the dialog.

Node properties

The following describes the input and output pins for the **Type Cast** node.

Pin	Description
Instance	The input value to be checked against the configured Type property.
Yes	The output pin that receives the execution pulse if the Instance is a Type .
No	The output pin that receives the execution pulse if the Instance is not a Type .
as ...	The output data pin that will contain the casted instance as Type , if the cast outputs Yes .

While

The **While** node provides a construct similar to a **For Loop** except the iteration of the loop is driven by an input boolean condition.

In the following example, we have an integer variable **x** that is initialized as 0. The while loop has a boolean condition that checks if **x** is less-than 10, and if so, will output an execution pulse to the **Repeat** pin, printing some text onto the screen and incrementing **x** by one. When **x** finally reaches 10, the boolean condition is no longer true, and the while loop ends by emitting an output execution pulse to the **Done** pin.

The **While** node intentionally does not support **break** semantics. Whether the loop breaks early should be controlled by the **Condition** input pin.

Node properties

The following describes the input and output pins for the **While** node.

Pin	Description
Condition	The condition that is evaluated on each loop iteration.
Repeat	The output pin that receives the execution pulse if the condition remains true .
Done	The output pin that receives the execution pulse if the condition is false .

Functions

A **Function** is a named unit of code that when executed, performs a specific task or intent. For example, calling `draw()` draws whatever arguments were specified while `print()` will print whatever is passed to the console.

In the context of Orchestrator, you can create user-defined functions within an **Orchestration**. These user-defined functions can be called from within that specific script from another graph, or could also be called from an external **Orchestration** script or even GDScript.

In this section, we'll talk about the various types of functions and how to use them within Orchestrator.

Function Types

Global scope functions In Godot, there is a category of functions called `@GlobalScope` functions. These are freestanding functions that take a set of input values and generate an output based on those inputs. Examples of global scope functions include `min`, `max`, or even `deg_to_rad`.

In Orchestrator, many of these appear until the **Math** or **Utilities** categories in the **All Actions** dialog.

The number of *input* and *output* pins, and their respective types, is driven by the Godot metadata.

Native functions In Godot, there are hundreds of native classes, which include things like `Node`, `Control`, or `Camera3D`. Each of these native classes expose a subset of their functions to the script subsystem, allowing for attached scripts to call these functions.

In Orchestrator, many of these appear in the **All Actions** dialog under the **Methods** or **Methods (Static)** categories. Under these two top-level categories, methods are cataloged by their class type. So finding something like rotating a `Node2D` around its Y-axis would be under **Methods** > `Node2D` > `rotate_y`.

The number of *input* and *output* pins, and their respective types, is driven by the Godot metadata.

Script functions In Godot, script-based functions are what users can freely write as part of Orchestrator or GDScript. These are functions you define yourself to define your own logic and behavior.

In the following sections, when we refer to user-defined functions, we're speaking about script-based functions.

The number of *input* and *output* pins, and their respective types, is driven by the function definition in the script.

Function access specifiers

In certain programming languages, a concept exists called *visibility*. In the simplest of terms, visibility defines who, what, and how that function can be called.

In Godot, there is no explicit concept of *visibility*. To this end, it means that all functions that are exposed to the scripting subsystem are public and can be called by any other script.

However, Godot has a general rule that functions that are intended to be private or not called by other objects, are defined with a prefix, `_` (underscore) character. If a function could be called by other objects, the underscore is omitted, denoting it's considered “public”.

While other engines like Unreal provide the ability to toggle function visibility, this is currently not available. Until Godot officially adds such support for this within its scripting API, there's no reasonable way to guard against this across the script landscape.

Declaring Script Functions

Creating a function

To create a Function within an orchestration:

1. In the **Component** panel, press the button in the **Functions** section.
2. Enter a name for the Function.

The Function will open in a new tab in the **Graph Editor**.

Editing a function

Once you have created a Function, you then need to define its functionality. This is a step-by-step process:

- Create the necessary input and output parameters.
- Add nodes to the function graph between the **Function Entry** and **Function Return** node.

Once the function is created, you modify the details in the **Inspector** view. To open the **Inspector** view to modify the function, you can:

- Select the **Function Entry** node in the graph.
- Select the **Function Result** node in the graph.
- Select your defined function in the **Functions** panel.

Inputs and outputs You can set the input and output parameters in the **Inspector** view for the Function.

To add an **Input** argument:

1. Press the + **Add Inputs** button in the inspector.
2. Given the newly added row a name instead of its default **NewParam**.
3. Change the argument's type, it defaults to **Variant**.
4. Specify whether the argument type is a single type, array, or dictionary.

Repeat this process until you've added all the desired input arguments.

To add an **Output** argument:

1. Press the + **Add Outputs** button in the inspector, if its enabled.
2. The return value is always labeled **Return Value** and cannot be changed.
3. Change the return value's type, it defaults to **Variant**.
4. Specify whether the argument type is a single type, array, or dictionary.

Any changes made to the input or output configurations will directly be reflected in the Function graph. Once the appropriate input and output arguments have been specified, the Function is now ready for you to define its functionality.

Godot's function API is limited to a single return value. If you need to return more than one value, consider wrapping the multiple values in an **Array** or **Dictionary**.

Coding

A Function is defined by creating an Orchestration graph that connects behavior between the **Function Entry** and the **Function Result** nodes. In this example, we will create the necessary nodes to apply the 3D version of Pythagorean Theorem, returning the distance between 2 points in 3D space.

```
gdscript title="Pythagorean's Theorem in GDScript" showLineNumbers
dx = (x2 - x1)^2 dy = (y2 - y1)^2 dz = (z2 - z1)^2 D = sqrt(dx +
dy + dz)
```

Representing this in an **Orchestration** may look like this:

1. Break the two **Vector3** positions (**start** and **end**) into their respective (**x1,y1,z1**) and (**x2,y2,z2**) values.
2. Calculate the values for **dx**, **dy**, and **dz**.
3. Add **dx**, **dy**, **dz**, and then take the **sqrt** of that value.
4. Return the result using a **Function Return** node.

Calling functions

Any built-in, native, or script/user-defined function can be called within your **Orchestration**. In this section, we're going to show how to call the **distance** function that was created in the above **Coding** section.

To create a node that will call your Function, you can either:

- Drag your Function from the **Components** view onto the graph, and select **Add Call to Function**.

- Right-click in the graph or drag from an appropriate execution or data pin to open the **All Actions** dialog. Search for your function, and select it to add the **Function Call** node.

The graph below takes two vector variables, and on every tick, calculates the distance between the two and pints it to the screen.

A script/user-defined function can be called from within the current **Orchestration**, another **Orchestration**, or even **GDScript**. For the two latter use cases, the script must have an object reference to call the functions. For the former, the implied **self** reference suffices.

Variable argument functions

Some functions in Godot are called variable argument functions, or more aptly *vararg* or *variadic* for short. These functions are designed to take zero or more static arguments, followed by zero or more variable arguments.

In many languages, variable argument functions are defined like:

```
gdscript title="Example of a variable argument function in GDScript"
showLineNumbers func min(x: Variant, y: Variant, ...): pass
```

In this example, the **min** function accepts at a minimum two **Variant** values called **x** and **y**. However, the function may be called with additional **Variant** values if the caller wishes to compute the minimum across more than two values.

Orchestrator supports all Godot built-in and native-class functions that use variable arguments. You can recognize these as the function call node will have a button in the bottom right corner, allowing you to add additional *input* pins to the node.

If you decide at a later time that you want to remove a variable argument from the node, simply right-click on the name of the argument and select **Remove Pin**.

Only pins that you were able to add to a variable argument function can be removed.

Orchestrator's own script functions currently do not support variable arguments. This will be added in a future update.

Method chaining

In text-based programming languages, method chaining is a technique to simplify code by calling multiple methods on the same object in a single line or a chain. In visual scripting tools, method chaining is not as common, but Orchestrator supports method chaining.

For example, in the following graph, we are applying a position and rotation to the camera.

You will notice that we have to not only call **Set Position** and **Set Rotation**, but we have to link the **Camera3D** node to each function call. If you were to add additional steps that you needed to perform on the Camera, you would begin to have spaghetti like connections.

Now, thanks to method chaining, you can connect the target output to the input of the next function call.

This makes the graph easier to read and understand, as well as easier to maintain.

To enable method chaining:

1. Left-click the **Call Function** node.
2. Open the **Inspector** view.
3. Check the **Chain** option.

Once enabled, a new output pin labeled **Target** will appear, allowing you to chain the functions together.

Input

Input is one of the most critical requirements in games, as it's used to drive all types of interactions with the game world, such as player movement. In Godot, it's preferred to use the **Project > Project Settings > Input Map** dialog to define mappings between input keys or sequences and actions that are triggered when those input keys or sequences are triggered.

Orchestrator works directly with the engine's **Input Map** using an **Input Action** node. The **Input Action** node is designed to return whether the action is triggered as an *output* pin, which can then be used as an *input* into other nodes that drive specific behavior.

Input action node that checks for action ui_up being pressed.

The **Input Action** node can provide you one of the several trigger modes. See the Input action trigger modes for more details.

Defining input mappings

Input mappings are defined in Godot using the **Project > Project Settings > Input Map** dialog. In this screen, you define the relationship between a key press or input sequence and an action that should be toggled on when that key press or input sequence is detected by the engine.

For example, to define an action called `move_forward` to be triggered when pressing the `W` key:

1. Open the **Input Map** tab by navigating the **Project > Project Settings** menu.
2. In the **Add New Action** field, type `move_forward`.

3. Press the **+** **Add** button to add the new action to the input map.
4. In the lower section of the dialog, find the action **move_forward**, and press the button.
5. In the **Event Configuration** pop-up dialog, press the **W** key followed by clicking the **OK** button.
6. A child row will be added to the action that reads **W (Physical)**.
7. Press the **Close** button to close the **Input Map** dialog.

Using an input mapping

Once an input mapping has been defined, you can use the **Input Action** node in an orchestration. The **Input Action** node checks the current state of the action defined in the input map, and returns either **true** if the action is triggered; otherwise, it returns **false**.

To add an **Input Action** node:

1. Right-click in the graph to open the **All Actions** dialog.
2. Search for **input action** in the search box.
3. Select the **Input Action** choice and either press the **Add** button or simply hit the **Enter** key.

By default, an **Input Action** node does not default to any specific action, so it's important to set the **Action** property in the **Inspector** view. See [Setting the input action name](#) for details.

Setting the input action name

The **Input Action** does not default to any specific action, so the action must be explicitly set in the **Inspector** view.

To set the action name:

1. Select the **Input Action** node in the graph.
2. Modify the **Action** property in the **Inspector** view by setting the action name, i.e. **move_forward**.

Input action trigger modes

Each **Input Action** node has a **Mode** property that describes precisely how the node should react to the action being triggered. For example, if you want to react only when the user pressed the key that triggers the action in the current frame but not in any future frames, you may want to select **Just pressed** rather than the default of **Pressed**.

The following describes each mode and how it's interpreted:

Input Action Mode	Description
Pressed	Returns true if the key or sequence that triggers the action is pressed.
Released	Returns true if the key or sequence that triggers the action is not pressed.
Just Pressed	Returns true if the key or sequence that triggers the action is pressed in the current frame only.
Just Released	Returns true if the key or sequence that triggers the action is released in the current frame only.

Input action properties

To set the properties for an **Input Action**:

1. Select the **Input Action** node in the graph.
2. Open the **Inspector** view and modify its properties.

Property	Description
Action	This is the user-defined action from Project > Project Settings > Input Map .
Mode	This defines how the action's triggering effect is recognized, whether pressed or released.

Math

Math or **Operator** nodes are essential to every game, allowing a script to do anything from moving an object each frame to computing the probability of specific scenarios based on a wide array of input characteristics. Godot, and by extension Orchestrator, provides nearly a dozen mathematical nodes to perform any type of computation.

Operators

Orchestrator classifies math operations into **5** key groups:

Type	Description
Boolean	Compares two values to determine their equality or relative comparison. <i>Examples: ==, !=, <, <=, >, or >=</i>
Numeric Math	Performs a unary operation on a single value or binary operation on two or more values. <i>Examples: +, -, *, /, -Unary, +Unary, %, or ^ (power).</i>

Type	Description
Bitwise	Performs bitwise operations on a given numeric value. <i>Examples: Shift left, Shift right, And, Or, Xor, or Negate.</i>
Logic	Performs a logical comparison between one or two values. <i>Examples: And, Or, Xor, and Not</i>
Contains	Checks whether a value is contained within a collection. <i>Examples: has or contains</i>

Legacy non-promotable operators

When **Enable Type Promotion** is disabled in **File > Settings > Editor > Behavior**, Orchestrator will register an action mapping for all possible combinations of operations between various types. For example, the node to add two **Integer** values differs from the node that adds two **Float** values.

For some users, they prefer this concrete style of operator nodes.

Orchestrations that use non-promotable operator nodes will continue to function as they always have, regardless of whether type promotion is or isn't enabled.

The non-promotable operator nodes are not currently deprecated, but they most likely will be in a future build. There are plans to remove the non-promotable flavors in Orchestrator 3.0, in favor of the new promotable types.

Adding nodes To add a math node to the orchestration, simply:

1. Right-click the graph to access the **All Actions** dialog.
2. Search for the math operation, for example **Multiply**.
3. Select the right math operation for the two operand types.

Changing operands After placing a **Math** node, if you need to change the operand type of one of the input values, then you must select a new node that matches your desired operands. By following the Add math nodes procedure, you can add the appropriate node and connect all the input/output pins before removing the old node.

To remove the old node, simply:

1. Select the node in the graph.
2. Either Right-Click and select **Delete** from the context-menu or simply press the **Del** button.

Promotable operators

When **Enable Type Promotion** is enabled (the default) in **File > Settings > Editor > Behavior**, Orchestrator registers a single operator node per operation. For example, there is a single **Utilities > Operators > Addition** node, regardless of its operands.

These new *promotable operator* nodes allow users to change the operand types on the fly. For some users, they prefer this approach, minimizing how often they need to access the **All Actions** dialog, duplicating an **Addition** node N times for Y operand combinations.

Unlike non-promotable operator nodes, these always spawn in with Any data types. These input operands can be changed by right-clicking the pin label and selecting **Change Pin Type**.

Finally, nodes like **Addition**, **Subtraction**, **Multiplication**, and **Division** all support 2+ operands. Normal operator precedence holds, operands are evaluated top-down from A to Z. To add additional operands, press the button.

Promotable operator nodes will automatically change their input pin type on the first connection to an input pin, when all inputs are currently wildcards. This provides for a better user experience and faster development, potentially minimizing at least one, if not both **Change Pin Type** context-menu usages.

Adding nodes To add a math node to the orchestration, simply:

1. Right-click the graph to access the **All Actions** dialog.
2. Search for the math operation, for example **Multiply**.
3. Select the **Utilities > Operators > Multiply** node.
4. Either connect a wire to one of the operands for its type to change, or manually select its type.

Changing operands Unlike non-promotable **Math** nodes, a promotable operator node's pin types can be changed at any time. Simply right click one of the pins, navigate to the **Change Pin Type** sub-menu and select the desired type.

When selecting the **Any** pin type, all pins will be returned changed to wildcard. This will trigger any active connections to the pins to be disconnected automatically. Additionally, duplicating a promotable operator node will set the pin types to **Any**, too.

Memory

Memory nodes are used to dynamically allocate and free objects in an **Orchestration**.

For example, you may add a **Button** to your user interface and based on some interaction, remove the button from the scene. In this case, you would use a **New** node to create the button and the **Free** node to free it.

These nodes are currently considered experimental .

It is okay to use these nodes, as they're plans to make these stable in Orchestrator 2.6. It's just their implementation may change in the future, but existing scripts will auto-upgrade.

Creating objects

The **New** or **Create Instance** node is responsible for creating a new instance of a given object. You can use this node to allocate a new instance of any type, as long as the type is not a singleton.

In languages like GDScript, the equivalent is the `new()` function: `gdscript title="Example of creating a button" showLineNumbers func _ready() -> void: var button = Button.new()`

In Orchestrator, the equivalent node is:

Create a new instance of a Button node

To create a **New** or **Create Instance** node:

1. Right-click in the graph to access the **All Actions** view.
2. Search for **New**, locating the **Memory > New** node.
3. Either press the **Add** button or simply hit **Enter**.
4. In the **Inspector** view, select the desired **Class Name** from the inspector, defaults to **Object**.

Godot's **Object** selector window shows all possible object class names; however, not all choices can be used. If the class name is a singleton, i.e. **OS**, this value will not be accepted, and you will need to select another value.

Node Properties

Property	Description
Class Name	Specifies the class name of the object to be created.
Instance	The newly created instance of the specified Class Name type.

Freeing objects

In languages like GDScript, you need to know the context of the object to pick precisely what type of function you should call to deallocate an object. For example, if the object is a **Node** and it's still in the scene tree, you should use `queue_free()`. If object is any other type of object, you should use `free()`.

In Orchestrator, we simplify this logic by using a universal **Free Instance** node. It's responsible for making the most appropriate choice for the deallocator function call based on the object's type. There's no need to guess.

Free an instance of a Button node

To create a **Free** or **Free Instance** node:

1. Right-click in the graph to access the **All Actions** view.
2. Search for **Free**, locating the **Memory > Free** node.
3. Either press the **Add** button or simply hit **Enter**.
4. Connect the object to be freed to the input **Target** pin.

Node Properties

Property	Description
Instance	The instance that should be deallocated or freed.

If the object has already been deallocated, this node is a no-op.

Properties

Properties are attributes on an object, that describe certain object state. For example, a **Button** in a user interface may have an icon property to provide some visual indicator of what the button does.

Property types

Properties are associated with a specific data type, such as **Boolean**, **Integer**, or **Float**, but can also be represented by more complex types such as **Object**, **Array**, **Dictionary**, or specialized packed array types. Properties are specifically color coded for easy identification, just like variables.

Accessing properties

Properties can be accessed in two ways: by using the **Get** node (called an accessor) or the **Set** node (called a mutator).

You can create a **Set** or **Get** node for a property by:

1. Right-clicking in the graph or dragging a connection from an existing node
2. In the **All Actions** dialog, search for the property of interest, i.e. **Get Position** or simply **Position**.
3. Make sure that the selection has an icon similar to and not on the right.
4. Either press the **Add** button or simply hit **Enter**.

Using **Property** nodes allows an **Orchestration** to read or modify the state of that property. For example, a character controller would update the **Position** property each frame based on input.

Get property

The property **Get** node accesses the current value of a property. These nodes do not have execution control flow pins as they're designed to specifically return a single value to be used as *input* values to other nodes.

Get Position property

In this example, to get the **position** property, the **Node3D** object reference must be provided as an input. This could be a **Get Self** reference if the current script is based on a **Node3D** type, or some other object.

Set property

The property **Set** node updates or modifies the value of a property. These nodes have execution control flow pins as they're designed to be called in a sequence chain. These nodes also have an *input* pin based on the property's type to set the property value with.

Set Position property

In this example, to set the **position** property, the **Node3D** object reference must be provided as an input. This could be a **Get Self** reference if the current script is based on a **Node3D** type, or some other object.

Properties via Methods

A property's value can be accessed or modified using method calls, too.

Get/Set Position using Methods

The benefit to using methods rather than properties is the implied **Self** reference.

Resources

Resources are the fundamental foundation to any game engine, and Godot is no different. Resources represent all things from assets such as images, textures, and sounds, to Godot-specific assets like scenes, scripts, and even **Orchestrations**. While many of these resources can be set directly on scene nodes using the **Inspector** view, you may need to dynamically load or work with resources at runtime.

For example, your game may define different themes that have unique styles, icons, textures that can be chosen in the game's settings by the user. In such cases, it may make sense to load only the preferred theme, and therefore these need to be handled dynamically at runtime.

In this section, you will learn more about **Resources**, and how to load and access them using Orchestrator.

Resource paths

A resource path is a special string that begins with **res://** and follows the Godot semantics for providing a universal resource identifier (URI) to the specific resource file. Orchestrator provides a specialized node, **Get Resource Path**, that specifically makes working with resource paths (URIs) simple.

Get Resource Path Node

To create a **Get Resource Path** node:

1. Locate the resource in the **FileSystem** view.
2. Select the file and drag the file onto the graph.
3. Upon releasing the mouse, select the **Get Path** option on the context-menu.

Node properties

The **Get Resource Path** node's can be modified in the **Inspector** view, after selecting the node.

Property	Description
Path	The complete Godot res:// path to the resource file in the project.

The **Get Resource Path** node does not load the resource from the **FileSystem**.

Instead, this node simply returns the fully-qualified path to the file in the Godot project. This can be useful for passing the resource's path to another node that loads the resource, such as a scene or an image.

Preload resources

When you need to load a resource like a texture at runtime, you could use the **Engine Singleton** node to access the **ResourceLoader** singleton. However, this approach loads the resource at the time the **Call Load** function is called in the execution flow, and if the resource loading blocks too long during the frame, the game may stutter.

One way to avoid this problem is use a technique called *preloading*, which is where resources are loaded automatically when the owning scene is loaded by the engine. This guarantees that the rendering of the scene is more stable and less likely to have stutter, while the trade-off is that the loading of the scene may take slightly longer.

Preload Resource Node

To preload resources with Orchestrator, you need to add a **Preload Resource** node:

1. Locate the resource in the **FileSystem** view.
2. Select the file and drag the file onto the graph.
3. Upon releasing the mouse, select the **Preload** option in the context-menu.

If the preloaded resource needs to change, repeat the above steps for the new resource.

When a **Preload Resource** node exists in an **Orchestration**, Orchestrator will automatically *load* the resource when the node that owns the orchestration enters the scene tree. Multiple subsequent nodes can accept the *output* pin as an input without incurring any overhead since the resource is loaded once.

There is discussion within the Godot community that **preload** may be deprecated. There is currently no exact timing or if this will happen, but if GDScript does consider this, Orchestrator will also follow.

While **preload** is a nice feature in certain cases, it introduces a certain level of complexity with resource loading that needs to be managed, that often complicated other areas of a script's lifecycle. It's generally recommended that users prefer using the **ResourceLoader** with threaded requests.

We plan to introduce several nodes in the future to simplify this so that users who rely on **preload** have alternatives without having to use the native **ResourceLoader** directly.

Scenes

Sometimes an **Orchestration** simply modifies the behavior of the node that it's attached; however, there are other situations where one node needs to coordinate, manipulate, or interact with other nodes in your scene. Orchestrator provides several useful ways to interact with other scene nodes and the Godot **SceneTree**.

Get self

While many nodes automatically default to **Self** for targets, this isn't universally the case. The **Get Self** node provides a way to effectively return a **self** reference, e.g.

Get self node

In other languages like C/C++ or Java, this is the equivalent to returning **this**.

Scene nodes

Godot provides dozens of scene node types, which provide some basic archetype in the engine. For example, there are nodes that provide user interface widgets, nodes that provide behaviors for 2D games, and others for 3D games. Additionally, these nodes have specializations for things such as characters, meshes, triggers, and more.

When you attach an **Orchestration** to a scene node, the **Orchestration** will inherit the behavior of the base type you assigned when you created the script. Most users assign the script's base type to match the node type you attach the orchestration to, a common parent type such as **Node3D**, **Node2D**, or **Node**.

Accessing scene node behaviors

Accessing a scene node's behavior (functions and properties) happens by searching for the function or property in the **All Actions** dialog. The main question is whether you want to access a function or property for the node that the **Orchestration** is attached, or are you wanting to access a function or property on another node in the scene.

Behaviors on script owner By default, the **All Actions** dialog presents a list of behaviors that are relevant to the **Orchestration** owner when you right-click on the graph. For example, if the base type selected is a **Node3D** and you would like to call the `rotate_y` function:

1. Right-click on the graph to open the **All Actions** dialog.
2. Search for `rotate_y`.
3. Select the **Call Rotate Y** and either pressed the **Add** button or hit **Enter**.

When you drag the mouse from either an input or output pin, the **All Actions** dialog presents a context-specific list of actions based on the type of pin you dragged from. This context-specific list often will be a subset or not even related to the base type of the **Orchestration**, so make sure to right-click on the graph to access the **All Actions** dialog if you're interested in the attached node's methods or properties.

Behaviors on other nodes To access behavior on other scene nodes, this requires a reference to that specific scene node.

Get a scene node

A reference to that node can be obtained simply by:

1. Selecting the desired scene node in the **Scene** view.
2. Drag the scene node onto the graph.
3. Releasing the mouse button will spawn a **Get Scene Node** node.

A **Get Scene Node** uses a Godot **NodePath** to refer to the scene node. If you rearrange your scene, the path to the node may change, and the **Get Scene Node** will need to be updated to reflect the new path. In order to change the path, select the **Get Scene Node** and then modify the **Node Path** property in the **Inspector** view.

We are aware that Godot introduced unique node ids in a recent Godot update. However, these unique node ids are not exposed to the scripting subsystem, and

cannot be used by Orchestrator at this time. Once they're available, we intend to transition to using those to make node path updates seamless.

Node properties

The **Get Scene Node** node is designed to use a **NodePath** to find the node in the current scene and return a reference to the physical node object in the scene. To modify the **Get Scene Node** properties, use the **Inspector** view.

Property	Description
Node Path	The relative path to the node that should be returned.

The *output* data pin on the node will contain the reference to the node identified by the **Node Path**. By dragging the mouse away from the *output* pin will open the **All Actions** dialog with access to context-specific actions to the scene node's type.

Scene tree

The **SceneTree** is the top-level concept in a scene, it represents the container that wraps the actual scene. It is also the place where you can access many common scene helper functions such as `create_timer`, `create_tween`, or calling the `quit()` function to exit the game.

There are two ways to access the **SceneTree**:

- Call the `get_tree()` method on the current node
- Use the **Get Scene Tree** visual script node

Regardless of which approach is used, dragging away from the *output* pin will provide direct access to functions and properties on the godot **SceneTree** object.

Using the `get_tree` method

The `get_tree()` method is accessible to an **Orchestration** as long as the base type you chose inherits from **Node**. If you chose a type that inherits from **Control**, **Node2D**, or **Node3D**, then you can safely access `get_tree()`.

1. Right-click the graph, opening the **All Actions** dialog.
2. Search for **Call Get Tree**, and select the **Call Get Tree** option by either pressing **Add** or hitting **Enter**.

Do not use attempt to use `get_tree()` during methods like `_init` or other early Godot callbacks. This is because until the node has entered the tree fully, `get_tree()` returns a null value, and will cause the script to error. For this reason, it's generally best to use the **Get Scene Tree** node.

Using the get scene tree node

Get Scene Tree Node

The **Get Scene Tree** node is another way to access the **SceneTree** from within an **Orchestration**. This method is preferred and will work regardless of the base type chosen.

1. Right-click the graph, opening the **All Actions** dialog.
2. Search for **Get Scene Tree** and select the **Call Get Scene Tree** option, either by pressing **Add** or hitting **Enter**.

Node properties

There are no properties that can be modified in the **Inspector** view. The **Get Scene Tree** node simply returns a reference to the **SceneTree** godot object as an *output* pin.

Instantiating scenes

You may find you have the need to load a specific scene and attach the scene to the current scene. This is often done when spawning in creatures, other players, or transitioning between scenes.

In languages like **GDScript** or **C#**, the process to instantiate a scene first involves loading the scene from disk, and then creating an instance of the scene. Orchestrator simplifies this in a single step using the **Instantiate Scene** node.

To add an **Instantiate Scene** node to the graph:

1. Right-click the graph and search for **Instantiate Scene**.
2. Select the **Instantiate Scene** option, and press **Add** or simply hit **Enter**.

Set what scene to load

Setting which scene to load can be done by either pressing the **Assign...** button on the node itself or by selecting the **Scene** property in the **Inspector** view. In both cases:

1. Navigate through the **Select File** dialog, locating the scene you instantiate.
2. Select the file and press **Open** or simply double-click the scene file to select.

Once the scene file is selected, the **Scene** property on the node and in the **Inspector** view will include the **res://** based path to the scene.

Node properties

To set the properties for the **Instantiate Scene** node, either modify the **Scene** property directly on the node or select the node and use the **Inspector** view to modify the **Scene** property.

Property	Description
Scene	Specifies the full res:// based path to the scene file that will be instantiated.

- The **Scene** property can be set dynamically by assigning an connection to the **Scene** input pin that contains the full path to the scene to load.
- The *output* pin will refer to the root **Node** in the scene that was loaded.

It is perfectly safe to connect the **Scene Root output** pin to other node input pins if necessary. The same instantiated scene root will be returned rather than instantiating the scene for each node input.

If used in a loop, a *new* scene will only be created when the *input* control flow pin is called.

Signals

Godot implements the observer pattern using signals. A signal represents a callback that any Godot **Object**, including an **Orchestration**, can connect to and react when the signal is fired. In addition, signals can provide contextual information using function arguments.

Creating signals

To create a new signal in an orchestration, follow these steps:

1. Press the button on the **Signal** component view.
2. Enter a unique name when the new signal entry is added. This is the name of the signal that parts of the orchestration as well as external Godot objects can connect and be notified when the signal is fired.

Renaming signals

To rename a signal:

1. Right-click the signal in the component panel, and select **Rename** in the context-menu.
2. Enter the new signal name in the text box and press **Enter**.

Any node in the current orchestration that refers to the signal will automatically update when the signal is renamed. Any other external resource, including another Orchestration will not automatically update and will need to be changed after the signal is renamed.

Deleting a signal

To delete a signal:

1. Select the **Signal** in the **Component** panel.
2. Right-click and select **Remove** in the context-menu.

Removing a **Signal** will also remove all signal nodes that refer to that signal from the orchestration.

Passing data with signals

Signals can also pass additional data to their listeners (objects that are connected to the signal), much like how a caller passes arguments to a function call. Each signal has a signature, much like a function, and when an **Object** connects to the signal, the function that is registered as the **Callable** for the signal must match the signal's argument signature.

For example, a **CheckBox** emits both a **pressed** and a **toggled** signal. The **pressed** signal does not pass any arguments, and so the callable will simply have an empty argument list. An orchestration function that listens to **pressed** signals would look like, taking note that there are no arguments:

```
func _on_pressed() -> void:
    pass
```

The **toggled** signal is different, it passes the current *toggled* state of the **CheckBox**, notifying the listener of the current checked state of the **CheckBox**. An orchestration function that listens to **toggled** signals would look like:

```
func _on_toggled(state: bool) -> void:
    pass
```

When you define a signal in Orchestrator, you can add arguments to the signal's signature in the **Inspector** view:

1. Select the **Signal** in the **Component** panel.
2. Press the + **Add Inputs** button in the **Inspector** view.
3. Set the name and type of your signal argument. For example, to pass a 3D object's position, you would set the type as **Vector3** and set the argument name as **position**.

The final result would look like this:

Emit a signal

When a signal is emitted or fired, this notifies any connected observers that some condition has occurred. The notification to all observer **Callable** functions happens within the main thread, synchronously.

Emitting a local signal

To emit a signal within an Orchestration:

1. Select the desired **Signal** from the **Component** view.

2. Drag the **Signal** onto the graph.
3. Releasing the mouse button will spawn an **Emit Signal** node.
4. Connect the execution pin with other nodes so that the signal is fired when needed.
5. If the **Signal** defines any arguments, connect any input arguments.

Emitting an external signal

To emit a signal defined within another Orchestration or native class:

1. Right-click the graph canvas to access the **All Actions** dialog.
2. Search for the desired signal name, e.g. `property_list_changed`
3. Select the desired signal and click the **Add** button or simply hit **Enter**.
4. Connect the execution pin with other nodes so that the signal is fired when needed.
5. If the `**Signal*` defines any arguments, connect any input arguments.

Changing signal's arguments

To adjust any signal properties, select the signal in the **Components** view and then modify the properties in the **Inspector** view.

Reducing the number of signal arguments will automatically disconnect any connections in the graph to that argument pin. In the event you accidentally reduce the number of arguments and break a connection, be sure to reconnect the pins after adjusting the argument count.

Await a signal

There are situations where you may need to make use of a coroutine, which allows the current execution of the code to temporarily yield until some condition is met before continuing the code continues where it left off. This technique is quite powerful as it simplifies the logic needed to yield and wait for that condition.

In Godot, coroutines work in conjunction with signals.

How to await a signal

To asynchronously wait for a signal to be fired:

1. Right-click the graph and search for **Await Signal** in the **All Actions** dialog.
2. Select **Await Signal** and press the **Add** button or simply press **Enter**.
3. Connect the execution pins with other nodes so that the *await* is called when needed.
4. Connect a reference object to the **Target** pin.
5. Connect, type, or select a signal from the drop-down when focusing the **Signal Name** input pin.

Node properties

There are no properties in the **Inspector** view that can be modified for the **Await Signal** node; however, there are two input pins that can be customized on the node in the graph.

Input	Description
Target	Specifies the object that the orchestration should connect to and wait.
Signal Name	Specifies the name of the signal that the target that the orchestration will wait.
Result	Returns the result of a signal callback, if a result was provided.

Singletons

A singleton represents a Godot object where there is only a single instance of that object present in your game. The Godot engine provides dozens of singletons for various purposes.

In addition, user code can also add and register their own singletons that can be accessed globally. To add a singleton reference to a script, place an **Engine Singleton** node onto the graph.

Singleton types

By default, Godot provides dozens of singletons, each providing access to very specific features. These singletons are registered by a unique name and can be referenced using that name.

The Godot provided singletons are:

Singleton Name	Description
AccessibilityServer	Server interface for screen reader support.
AudioServer	Server interface for low-level audio access.
CameraServer	Server keeping track of different cameras accessible in Godot.
ClassDB	A class information repository.
DisplayServer	A server interface for low-level window management.
EditorInterface	Godot editor's interface.
Engine	Provides access to engine properties.
EngineDebugger	Exposes the internal debugger.
GDEXTensionManager	Provides access to GDEXTension functionality.
GDScriptLanguageServer	GDScriptLanguageServer - Experimental.
Geometry2D	Provides methods for some common 2D geometric operations.

Singleton	
Name	Description
Geometry3D	Provides methods for some common 3D geometric operations.
IP	Internet protocol (IP) support functions such as DNS resolution.
Input	A singleton for handling inputs.
InputMap	A singleton that manages all InputEventActions .
JavaClassWrapper	Provides access to the Java Native Interface (JNI).
JavaScriptBridgeSingleton	Singleton that connects the engine with the browser's JavaScript context in Web export.
Marshalls	Data transformation (marshalling) and encoding helpers.
NativeMenu	A server interface for OS native menus.
NavigationMeshHelper	Helper class for creating and clearing navigation meshes.
NavigationServer2D	Server interface for low-level 2D navigation access.
NavigationServer2DManager	A singleton for managing NavigationServer2D implementations.
NavigationServer3D	Server interface for low-level 3D navigation access.
NavigationServer3DManager	A singleton for managing NavigationServer3D implementations.
OS	Provides access to common operating system functionalities.
Performance	Exposes performance-related data.
PhysicsServer2D	A server interface for low-level 2D physics access.
PhysicsServer2DManager	A singleton for managing PhysicsServer2D implementations.
PhysicsServer3D	A server interface for low-level 3D physics access.
PhysicsServer3DManager	A singleton for managing PhysicsServer3D implementations.
ProjectSettings	Stores globally-accessible variables.
RenderingServer	Server for anything visible.
ResourceLoader	A singleton for loading resource files.
ResourceSaver	A singleton for saving Resources to the filesystem.
ResourceUID	A singleton that manages the unique identifiers of all resources within a project.
TextServerManager	A singleton for managing TextServer implementations.
ThemeDB	A singleton that provides access to static information about Theme resources used by the engine and by your project.
Time	A singleton for working with time data.
TranslationServer	A server responsible for language translations.
WorkerThreadPool	A singleton that allocates some Threads on startup, used to offload tasks to these threads.
XRServer	Server for AR and VR features.

Selecting the singleton

There are two unique ways to place a **Singleton** node into the graph.

The first approach is to search for the **Singleton** by name in the **All Actions**

dialog. For example, if you want to place the **Time** singleton, simply search for **Time**:

The other approach is to place an **Engine Singleton** node into the graph. The singleton node will automatically default to the **Engine** singleton. To change which singleton you want to access:

1. Select the **Singleton** node in the graph.
2. Select the designed choice in the **Inspector** view for the **Singleton** property.

Accessing singleton functions

Godot singletons provide access to commonly used functions, such as `Time.get_unix_time_from_system()`. To access singleton functions, simply:

1. Drag the mouse away from the output pin on the **Engine Singleton** node.
2. When releasing the mouse, the **All Actions** dialog will show.
3. Search for the desired function, i.e. `get unix time from system`
4. Select the function and press **Add** or hit **Enter** to place the function onto the graph.

Registering a user-defined singleton

In addition to the singletons that are provided by Godot, users can also add their own singletons. When a singleton is added, it's expected that you guarantee that the instance remains valid until the singleton is unregistered. If a registered singleton is deleted before it's removed, the editor or game will likely crash.

To add a user-defined singleton using **Orchestrator**:

1. Place an **Engine Singleton** node that refers to the **Engine** singleton onto the graph.
2. Drag the mouse away from the output pin.
3. In the **All Actions** dialog, search for `register` and select **Call Register Singleton**.
4. Provide a unique name to register the singleton instance with, i.e. `MySingleton`.
5. With a reference to your singleton object, connect the reference to the **instance** input pin.

User-defined singletons are registered dynamically at runtime, and therefore are not accessible using the **Engine Singleton** node. To access a user-defined singleton, use the **Engine Singleton** node calling the **Call Get Singleton** function to obtain a reference at runtime.

For most cases, it may be simpler to use an **Autoload** node instead, since autoloads are registered in the **Project Settings** and are directly selectable in the **Inspector** view when selecting the **Autoload** node. You also won't

need to worry about unregistering the **Autoload** as Godot will handle this automatically.

To unregister a user-defined singleton using Orchestrator:

1. Place an **Engine Singleton** node that refers to the **Engine** singleton onto the graph.
2. Drag the mouse away from the output pin.
3. In the **All Actions** dialog, search for **unregister** and select **Call Unregister Singleton**.
4. Provide the registered singleton's name in the **name** input pin.

Utilities

Make built-in type nodes

For built-in Godot types like **Vector3**, you do not use the **Memory > New Instance** node, but instead use the **Make** or **Compose** node. This is because built-in types are allocated on the stack rather than on the heap.

These nodes allow you to create a specific built-in type from one or more different constructor arguments. For example, **Vector3** provides the following choices:

Make Vector3 Actions

One of the most common use cases is where you want to supply the X, Y, and Z values directly:

Make Vector3 Node using X, Y, and Z

This is the equivalent of GDScript: `gdscript title="Creating a Vector3" showLineNumbers var vec : Vector3 = Vector3(1.0, 2.0, 3.0)`

Breaking built-in type nodes

When you already have a **Vector3**, you may be interested in obtaining only the X coordinate value. This is where the **Break** node is useful, as it allows you to take a struct-based built-in type and decompose it to its lower component types.

There is generally only one **Break** node per built-in type, e.g. for **Vector3**:

Break Vector3 into X, Y, and Z

Once the built-in type has been broken, you can access the individual components independently.

This is the equivalent of GDScript: `gdscript title="Accessing a Vector3 X component" showLineNumbers var vec : Vector3 = Vector3(1.0, 2.0, 3.0) # Where you only want to print the X component of the Vector print(vec.x)`

Print string

The **Print String** node extends the standard Godot print functions, allowing to add text to an on-screen overlay.

Print String Node

The icon indicates the node only operates during debug/editor builds. This makes this extremely useful to debugging without needing to remove the nodes for exports.

Node properties

The following describes the input and output pins for the **Print String** node.

Property	Description
Text	Specifies the text to be printed.
Print To Screen	Specifies whether the text will be added to the PrintString overlay canvas.
Print To Log	Specifies whether the text will be added to the console/output panel.
Text Color	The color to be used when adding the text to the PrintString overlay canvas.
Duration	The time that the text remains on the screen before it disappears.

Variables

Variables are properties that hold a value or reference to an Object in your game. These properties can be accessible internally to the **Orchestration** that contains them, or can accessible externally so that their values can be modified by designers working with instances of the orchestration attached on scene nodes.

Variable Types

Variables can be created using a variety of different types, including data types such as Boolean, Integer, or Float, but can also reference types for holding more complex values like Objects, Nodes, Resources, or other named types. Arrays, Dictionaries, and specialized Packed-Arrays can be created too.

Built-in types

Each type is specifically color coded for easy identification:

Variable Type	Color	Icon	Represents
Boolean			True or false value (<code>bool</code>).
Integer			Signed 64-bit value between -2^{63} and $2^{63}-1$.
Float			Signed 64-bit double-precision floating-point number.
String			Group of alphanumeric characters, such as <code>Hello World</code> .
StringName			An immutable string that is faster to compare than <code>String</code> .
Rect2			2D axis-aligned bounding box using floating-point coordinates.
Rect2i			2D axis-aligned bounding box using integer coordinates.
Vector2			2D vector using floating-point coordinates.
Vector2i			2D vector using integer coordinates.
Vector3			3D vector using floating-point coordinates.
Vector3i			3D vector using integer coordinates.
Vector4			4D vector using floating-point coordinates.
Vector4i			4D vector using integer coordinates.
Transform2D			2x3 matrix representing a 2D transformation.
Transform3D			3x4 matrix representing a 3D transformation.
Plane			A plane in Hessian normal form.
Quaternion			Represents 3D rotations.

Variable Type	Color	Icon	Represents
AABB			3D axis-aligned bounding box.
Basis			3x3 matrix for representing 3D rotation and scale.
Projection			4x4 matrix for 3D projective transformations.
Color			RGBA color.
NodePath			Pre-parsed scene tree path, can be relative or absolute.
RID			Handle for a Resource 's unique identifier.
Object			Base class for all classes in the Godot engine.
Dictionary			Container that holds key-value pairs.
Array			Container that holds a sequence of elements.
Callable			Represents a method or a standalone function callback.
PackedByteArray			Packed array of byte values.
PackedStringArray			Packed array of String values.
PackedInt32Array			Packed array of 32-bit integer values.
PackedInt64Array			Packed array of 64-bit integer values.
PackedFloat32Array			Packed array of 32-bit floating-point values.
PackedFloat64Array			Packed array of 64-bit floating-point values.
PackedVector2Array			Packed array of Vector2 values.
PackedVector3Array			Packed array of Vector3 values.
PackedColorArray			Packed array of Color values.

Connection colors can be adjusted in the Orchestrator tab, **File > Settings > Interface > Theme** section.

Arrays and dictionaries

In the **Select Variable Type** dialog, the option for **Array** or **Dictionary** is intentionally omitted. This is because these container types are defined as part of the drop-down next to the variable type.

Icon	Description
	Specifies that the variable type is not an Array or Dictionary container type.
	Specifies that the variable type is the element type of a typed Array .
	Specifies that the variable type before the container drop-down is the Dictionary key, the type afterward is the value.

When the **Array** element or the **Dictionary** key and value set to **Variant**, the **Array** or **Dictionary** is untyped.

Exported variables

For a variable to have instance-specific state managed by the scene, it must be **exported**.

Variables in the component panel are not exported by default, using the . By clicking the eye icon or selecting the variable and toggling the **Exported** property in the **Inspector** view, the variable will be marked as exported using the icon .

Once a variable has been exported, if the orchestration is attached to any scene nodes, the value of the variable can be initialized for each specific scene node that the orchestration is attached separately.

This allows for creating orchestrations with unique behaviors and reusing that behavior on different nodes, but allowing each node to work differently based on unique per-node configurable values.

Private variables

In Godot, there is no formal concept of **visibility**. This means that a variable is never really “private” so that it cannot be modified by another script.

There is a general rule that using `_` (underscore) prefixes identifies a variable that is considered private, but to the scripting subsystem, that will still be visible.

Variable descriptions

A variable can have a unique description, that provides information about its purpose, intent, use, or whatever other details you need. The description is provided as part of the **Component** view tooltip and as part of the tooltips on all **Get** or **Set** variable nodes in the script.

Additionally, these descriptions are made available to the EditorHelp for the script file or class, depending on how the script is defined.

Promoting to variable

Variables can also be created automatically by using the **Promote to Variable** context-menu option for pins on graph nodes. Right-click any input or output data pin on an orchestration node and select **Promote to Variable**.

For input pins, this will also spawn a **Get** variable node, while for output pins this will spawn a **Set** variable node.

Accessing variables

When working with variables in an orchestration, you will access them in one of two ways: by using the **Get** node (referred to as a Getter or Accessor) to get the value of the variable or by using the **Set** node (referred to as a Setter or Mutator) to set the value of the variable.

Get the value of variable speed

Set the value of variable speed

You can create a **Set** node or a **Get** node for a variable by right-clicking in the graph and typing either **Set (variable name)** or **Get (variable name)**. Another method is to drag and drop the variable onto the graph canvas, selecting to spawn either a **Set** or **Get** node.

Editing variables

Variable values can be set either as part of the orchestration's node graph or as a default value in the **Inspector** view. When setting a default value on an *exported* variable, the default value will be overridden if the scene node that the orchestration is attached specifies a custom value in the **Inspector**. To set variable default values:

1. Click the variable in the component panel.
2. In the **Inspector** view, modify the value associated with the **Default Value** property.

Not all variable types allow specifying default values. For example, if the variable represents a **Node** or a Godot **Object** type, then the default value property won't be available.

Renaming variables

To rename a variable:

1. Right-click the variable in the component panel, and select **Rename** in the context-menu.
2. Enter the new variable name in the text box and press **Enter**.

Any **Get** or **Set** node already in any graph will automatically update when the variable is renamed.

Variable properties

You can set all the properties for a variable in the **Inspector** panel. Some variables may have more or less properties than others, depending on the variable's type.

Property	Description
Name	The name of the variable, can only be changed from the component panel.
Category	The logical group the variable belongs. Setting this to an empty string or Default implies the variable is uncategorized.
Constant	Specifies whether the variable should be treated like a constant and be immutable.
Exported	Toggles whether the variable's value can be managed from within the scene its attached.
Variable Type	Specifies the variable type and container type if its an Array or Dictionary .
Default Value	For many variable types, sets an initial, default value assigned to the variable when the orchestration is created.
Description	Specifies a custom description that describes the variable, its purpose, and usage.

Getting and setting variable values

Variables can also be edited as part of your orchestration graphs by using the **Get** and **Set** nodes. The easiest way to create them is by dragging a variable from the component panel onto the graph. A small context-menu will appear asking if you would like to create a **Get** or **Set** node.

Get nodes

A variable **Get** node returns the variable's value to in the orchestration. Once created, these nodes can be plugged into any node that takes the appropriate variable type.

Set nodes

A variable **Set** node changes the value stored in the variable. Note that these must be called by an execution pin in order for the node to execute and set the variable's value.

Shortcuts when dragging

- **Ctrl-drag:** Creates a **Get** node
- **Alt-drag:** Creates a **Set** node